

Connecting biological data with AI systems via MCP

Concepts, design patterns, and implementation

How are we?

Damian Szklarczyk
Qingyao Huang
David Patsch
Radja Hachilif



Part of System Biology Group at University of Zurich led by
Christian von Mering

Projects/databases we are working on:
STRING, PaxDB, MicrobeAtlas, eggNOG

Part 1: Workflow

Part 2: The Protocol

Part 3: Grounding

Part 4: Tools

Part 5: Metadata

Part 6: Deployment / Hands-on

Before we start...

Version: 12.5 (PREVIEW)

DAMIAN SZKLARCZYK | LOGOUT | SURVEY



Search | Download | Help | My Data

Welcome to STRING

Protein-Protein Interaction Networks
Functional Enrichment Analysis

ORGANISMS
12535

PROTEINS
59.3 mio

INTERACTIONS
>20 bln

SEARCH

Protein-protein interaction network

Type: Physical

Species: Homo sapiens

Number of edges: 70,797

Number of nodes: 11,442

Score cut-off: 0.900

© STRING CONSORTIUM 2026

 SIB - Swiss Institute of Bioinformatics

 CPR - Novo Nordisk Foundation Center Protein Research

 EMBL - European Molecular Biology Laboratory

ABOUT

Content

References

People

Statistics

INFO

Scores

Use scenarios

FAQs

Cookies/Privacy

ACCESS

Versions

APIs

Licensing

Usage

CREDITS

Funding

Datasources

Partners

Software

 GLOBAL
CORE
BIODATA
RESOURCE

 elixir
Core Data
Resource

STRING is a Core Data Resource as designated by [Global Biodata Coalition](#) and [ELIXIR](#).

<https://string-db.org>

[Protein by name](#) >[Multiple proteins](#) >[Proteins by sequences](#) >[Proteins with Values/Ranks](#) >[Protein families \("COGs"\)](#) >[Pathway / Process / Disease](#) >[Add organism](#) >[Organisms](#) >[STRING chat](#) >[Examples](#) >[Random entry](#) >

SEARCH

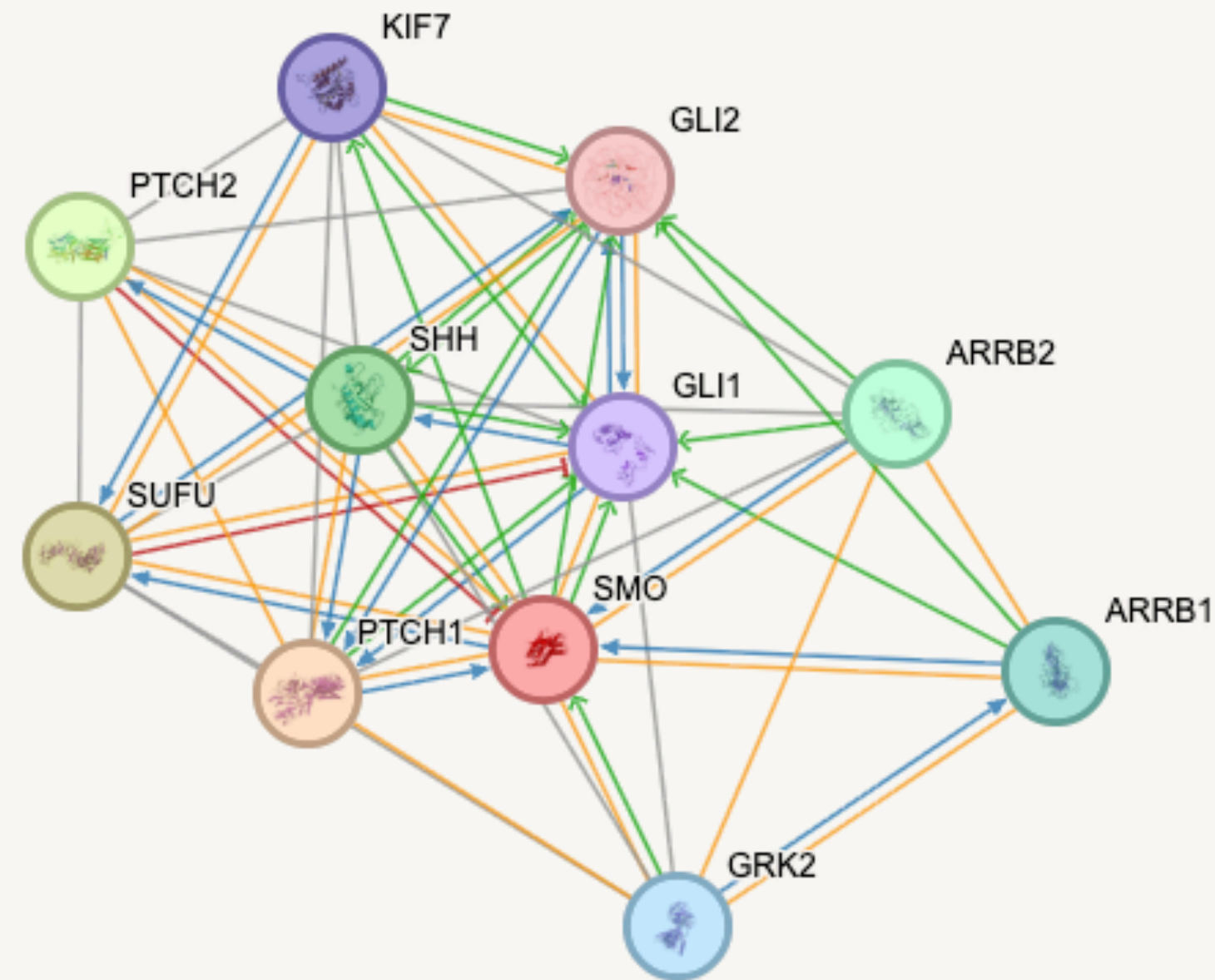
Single Protein by Name / Identifier

Protein Name: (examples: [#1](#) [#2](#) [#3](#))

Organisms:

[Advanced Settings](#)

SEARCH



Viewers >

Legend >

Settings >

Analysis ▾

Exports >

Clusters >

+ More

- Less

Network Stats

number of nodes: 11
 number of edges: 45
 average node degree: 8.18
 avg. local clustering coefficient: 0.858

expected number of edges: 14
 PPI enrichment p-value: 3.53e-11
your network has significantly more interactions than expected (what does that mean?)

Functional enrichments in your network

Note: some enrichments may be expected here (why?)

[explain columns](#)

> Biological Process (Gene Ontology)					
GO-term	description	count in network	strength	signal	false discovery rate
GO:0008589	Regulation of smoothened signaling pathway	7 of 95	2.12	4.29	3.22e-10
GO:0007418	Ventral midline development	4 of 6	3.08	3.99	5.19e-08
GO:0009953	Dorsal/ventral pattern formation	6 of 98	2.04	3.34	5.19e-08
GO:0021513	Spinal cord dorsal/ventral patterning	4 of 20	2.55	3.06	1.71e-06
GO:0021904	Dorsal/ventral neural tube patterning	4 of 25	2.46	2.93	2.54e-06
(more ...)					

STRING chat

This is a natural language interface to access and explore STRING data, backed by its MCP server.

You can also use the [remote MCP server](#) directly in your own AI tools and bioinformatics apps.

Ask about proteins, interactions, functions...



Example questions >>

Note: This chat uses data from the **STRING** protein interaction database, but AI-generated responses may also include inferred or external knowledge. For reliable citation and reproducibility, always request a direct STRING link and verify results from the source. For details on how your data is handled, see our [Privacy Policy](#). If you have any questions, please [contact us](#).

DEMO

Basic nomenclature:

Agent: This is your LLM that answers the questions, usually a GPT surrounded by an controller, embedded in some environment.

Also: **LLM, GPT, Chat, Model, Assistant**

MCP Server: Model Context Protocol Server. A server running your code (tools) that communicates via MCP with agent.

Also: **Server, MCP**

Tool: Code that returns a structured response for a query, ran by the controller or by an MCP. Could be simple API call, or a call to the database.

API: Application Programming Interface. Common, structured, way that the tool can communicate with the outside resources via network.

Also: **Database, Service, REST Interface, Resource**

Agentic vs. non-agentic

It's a spectrum

Non-agentic system: A mostly pure LLM system. It primarily processes the information provided in its context and generates a response. It cannot independently process large datasets or act on complex data outside its context. Most basic LLM API usage falls into this category. However most chat interfaces have some degree of agency including execution loops.

Agentic system: An LLM-based system that can autonomously select and use tools to accomplish a task, for example, reading and writing files, executing code, or querying external resources. It can operate in a multi-step execution loop, inspect intermediate results, and decide what to do next.

PART 1

Workflow

Some knowledge can live inside the model

QUESTION

When did the Second World War begin?

ANSWER

1 September 1939

Current information cannot be frozen into model weights

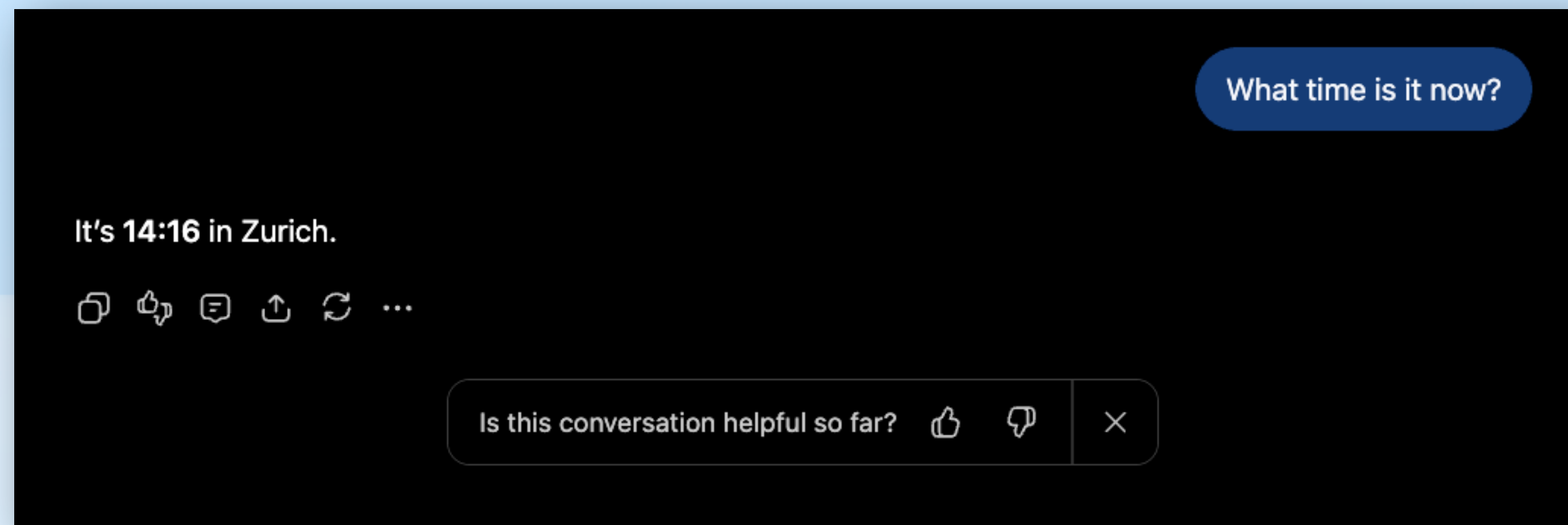
QUESTION

What time is it in Zurich now?

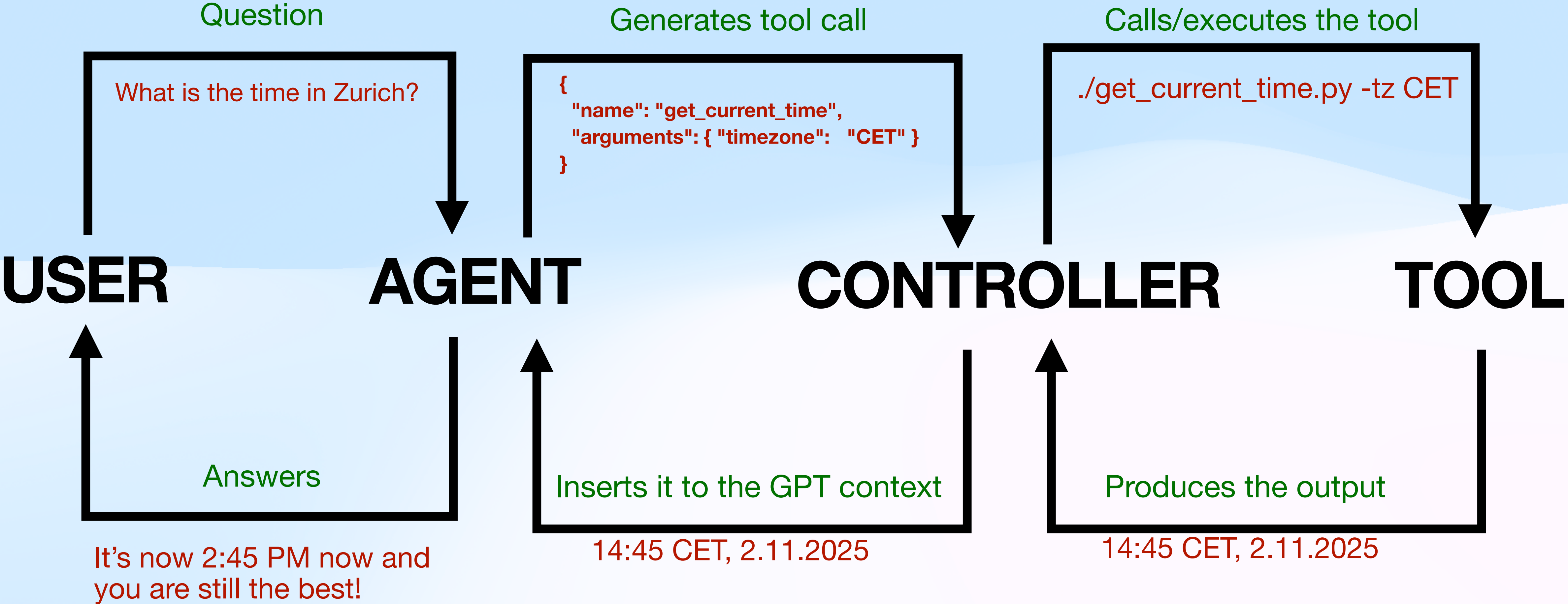
ANSWER

???

But it somehow works...



GPT have access to tools...



GPT tool calls

USER: What time is in Zurich?

ASSISTANT:

```
<TOOL_CALL>  
get_time({"timezone":"Europe/Zurich"})  
</TOOL_CALL>
```

```
CONTROLLER:  
<TOOL_RESULT>  
15:03  
</TOOL_RESULT>
```

ASSISTANT:

It is 15:03 in Zurich.

Lots of GPT functionality is based on these tools:

- **Image generation**
- **Current time**
- **Calculator**
- **Code execution**
- **Weather**
- **Memory**
- **File system access**
- **Reading files**
- **...**

**This list is supplied on GPT initialization.
They are trained to preferentially use these.**

Tool registry:

```
tools = [  
  {  
    "name": "get_current_time",  
    "description": "Returns the current time for a given timezone.",  
    "parameters": {  
      "timezone": {"type": "string", "description": "IANA timezone name"}  
    }  
  },  
  {  
    "name": "web_search",  
    "description": "Searches the web for recent information.",  
    "parameters": {  
      "query": {"type": "string", "description": "Search terms"}  
    }  
  }  
]
```

Some knowledge can live inside the model

QUESTION

When is a function of TP53 in a cell?

ANSWER

TP53 helps to regulate cell growth...

Detailed information cannot be frozen into model weights

QUESTION

List all GO BP terms associated with TP53

ANSWER

[some list]

Tool registry:

```
tools = [  
  {  
    "name": "get_GO_terms_for_protein",  
    "description": "Returns all the go terms association with the protein.",  
    "parameters": {  
      "uniprot_ac": {"type": "string", "description": "UniProt identifier of protein"}  
    }  
  },  
  {  
    "name": "get_proteins_for_GO_term",  
    "description": "Returns all proteins association with a given go term.",  
    "parameters": {  
      "query": {"type": "string", "description": "GO term identifier"}  
    }  
  }  
]
```

MCP (Model Context Protocol... Server)

- Exposes available tools to the agent
- Executes tool code on controller request ❤️
- Processes and structures returned data ❤️
- Sends formatted results back to the controller

openAPI

- **Exposes available tools to the agent** 🙋
- **Executes tool code on controller request** 💩
- **Processes and structures returned data** 💩
- **Sends formatted results back to the controller**

openAPI



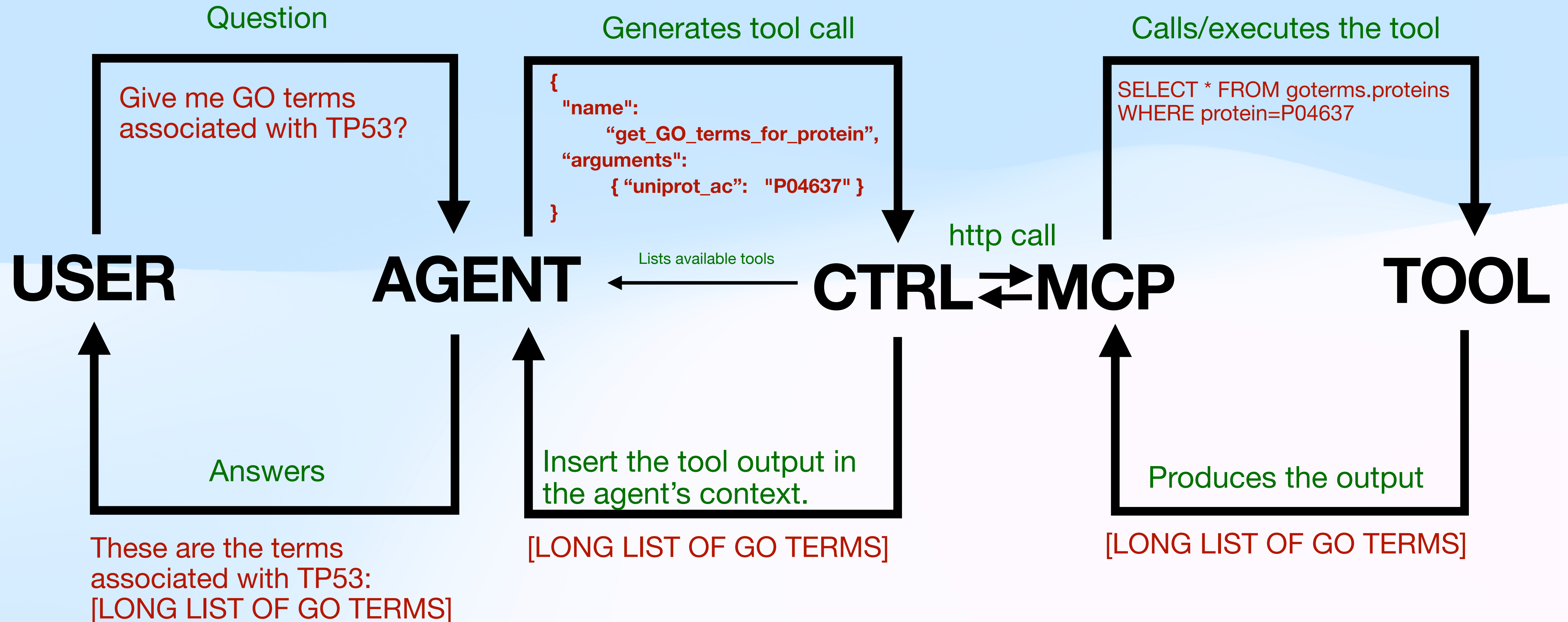
2 min vibing coding...

MCP Server

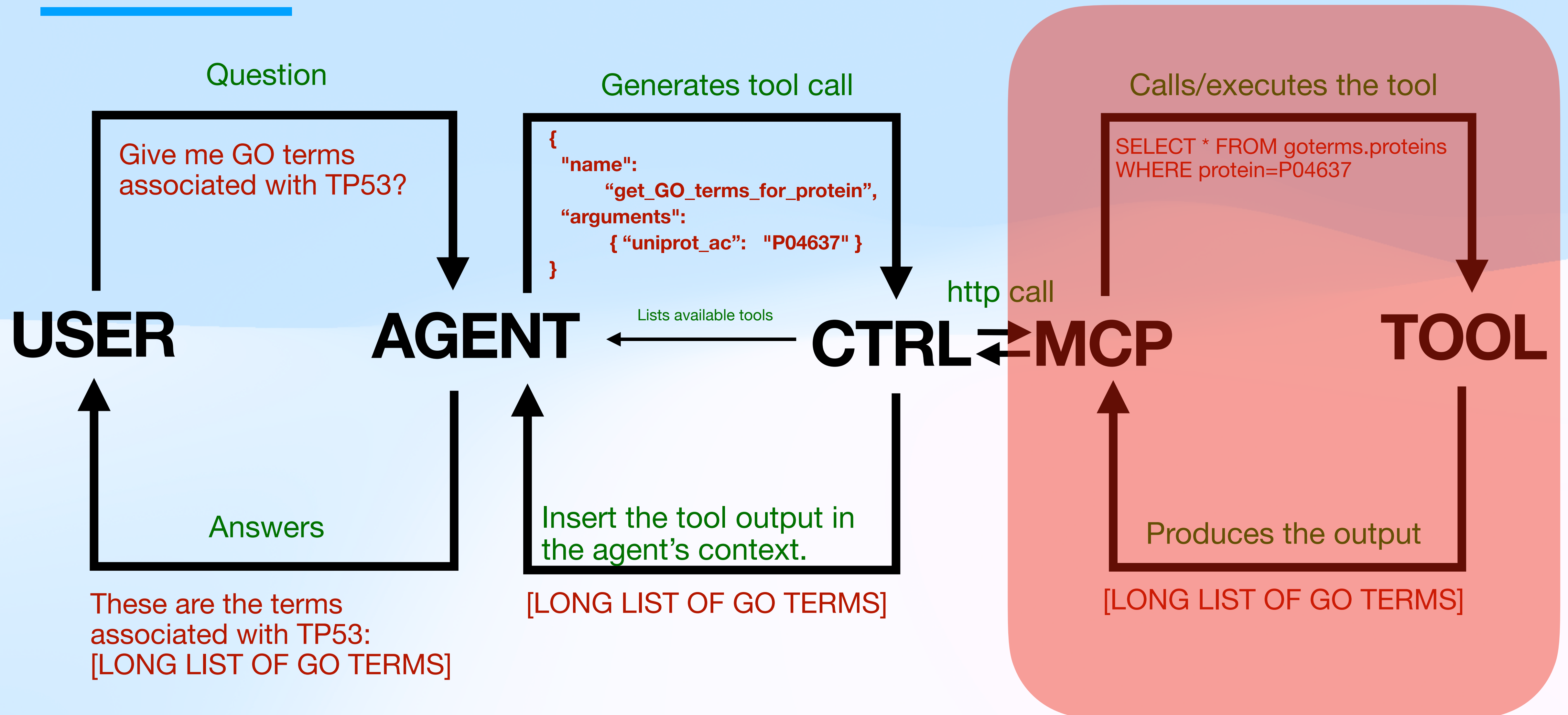
MCP (Model Context Protocol... Server)

- Exposes available tools to the agent
- Executes tool code on controller request ❤️
- Processes and structures returned data ❤️
- Sends formatted results back to the controller

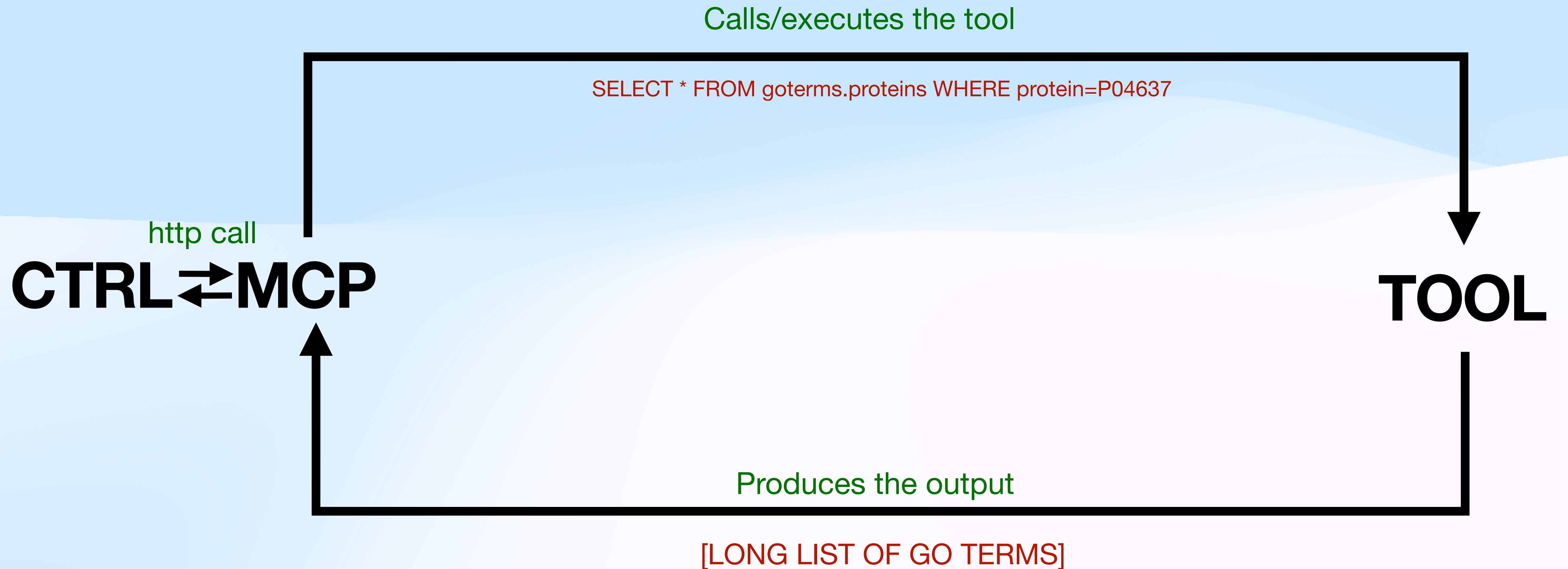
Agent-MCP workflow...



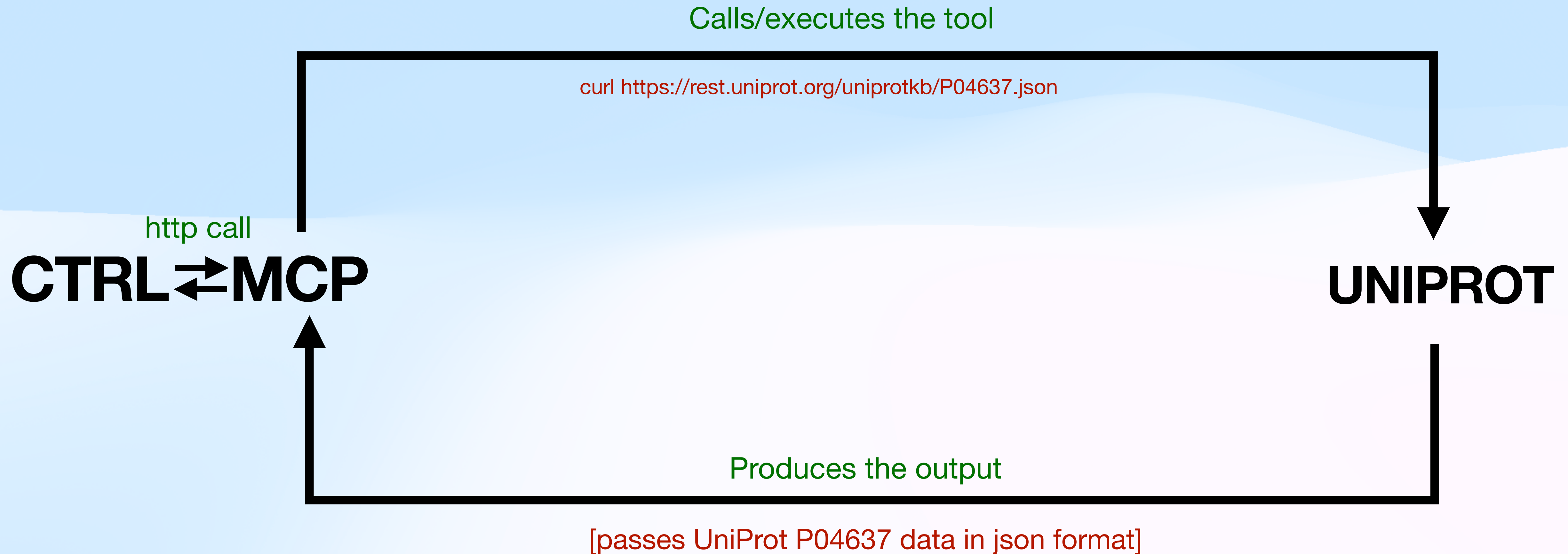
Agent-MCP workflow...



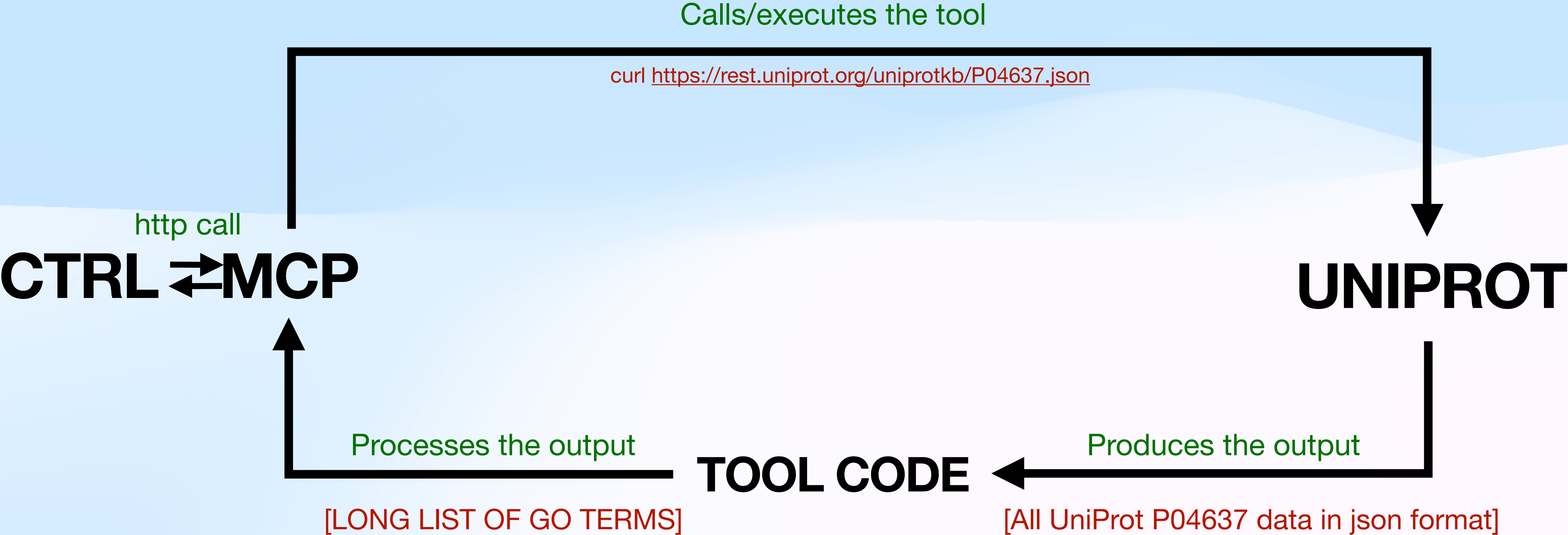
MCP workflow...



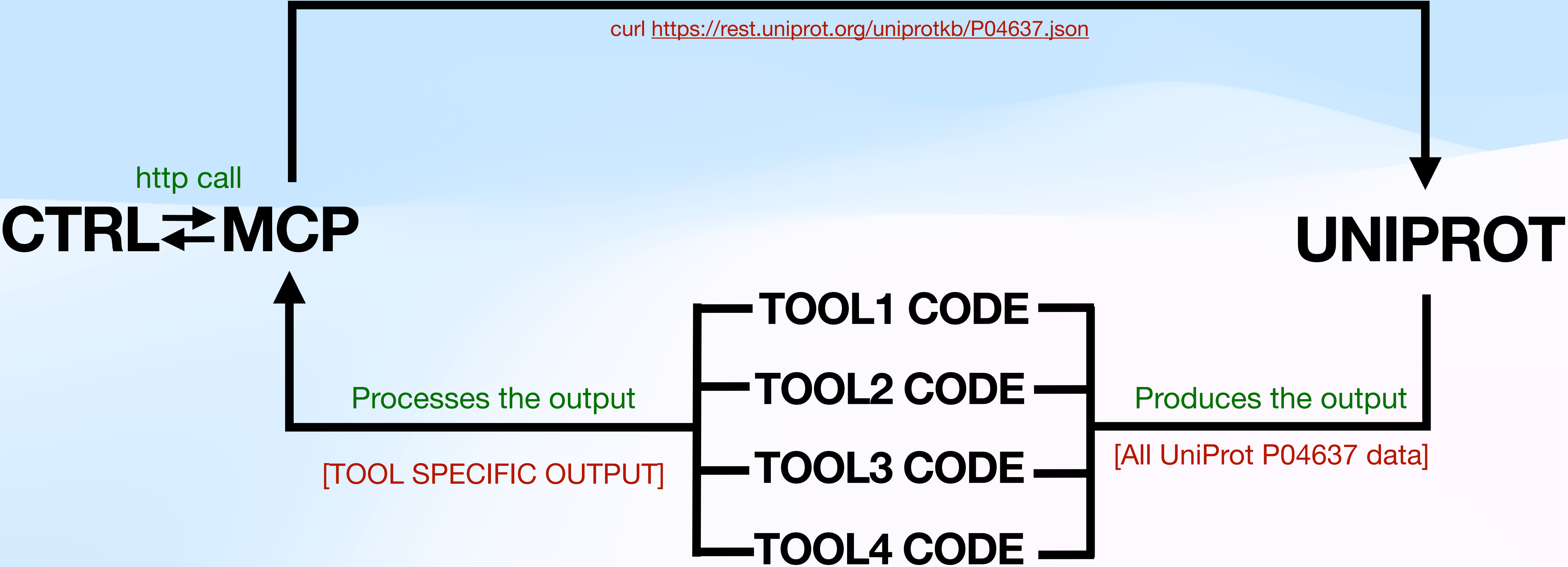
MCP workflow...



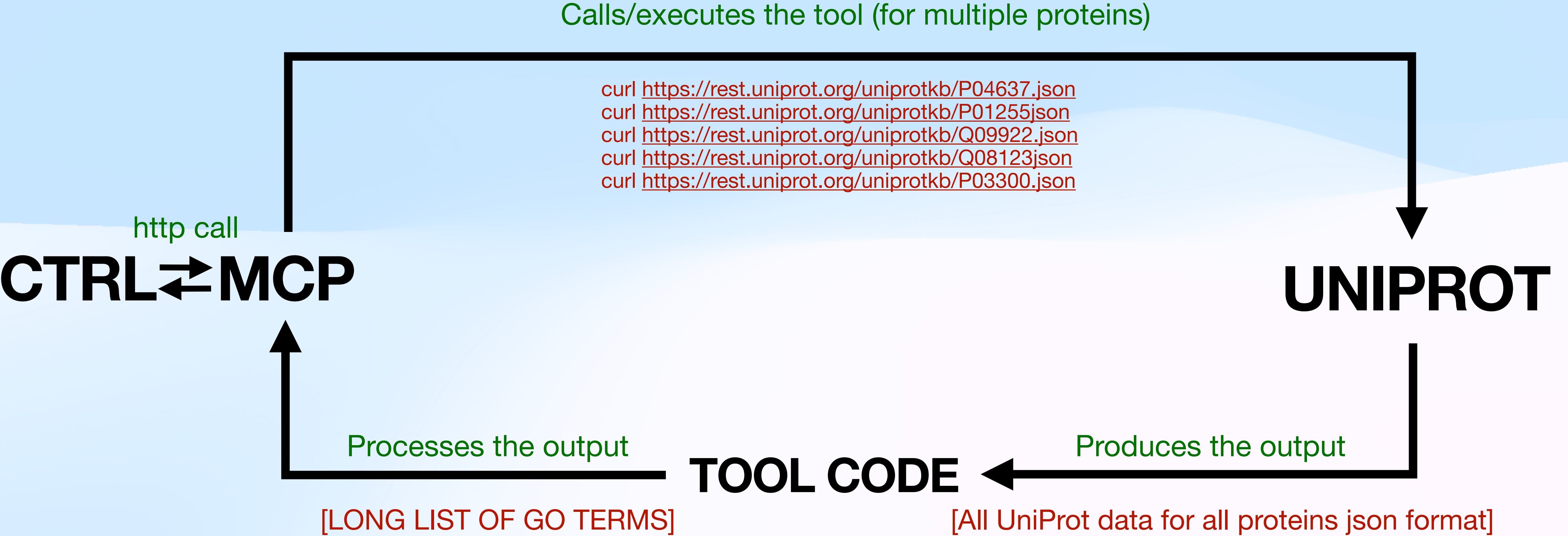
MCP workflow...



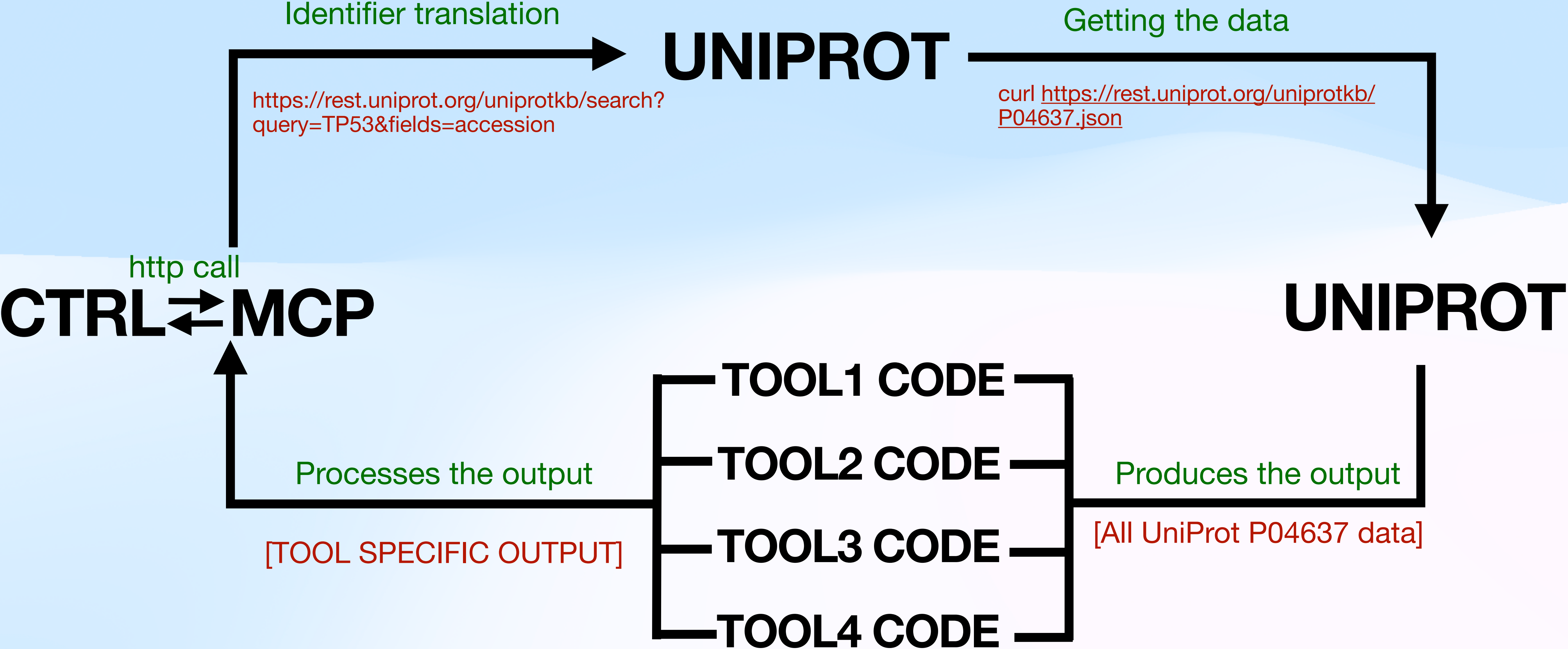
MCP workflow...



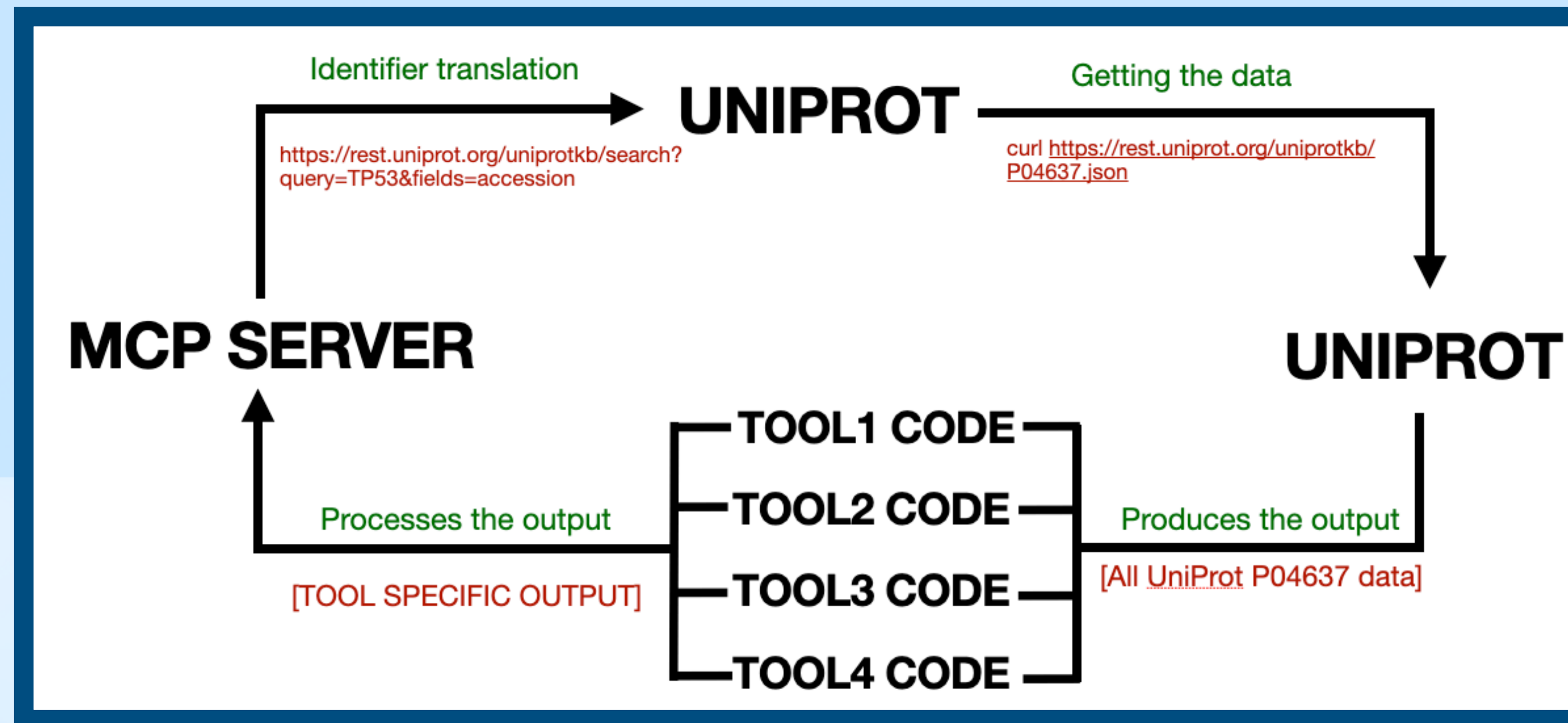
MCP workflow...



MCP workflow...



Remote MCP Server



<https://mcp.example.org>

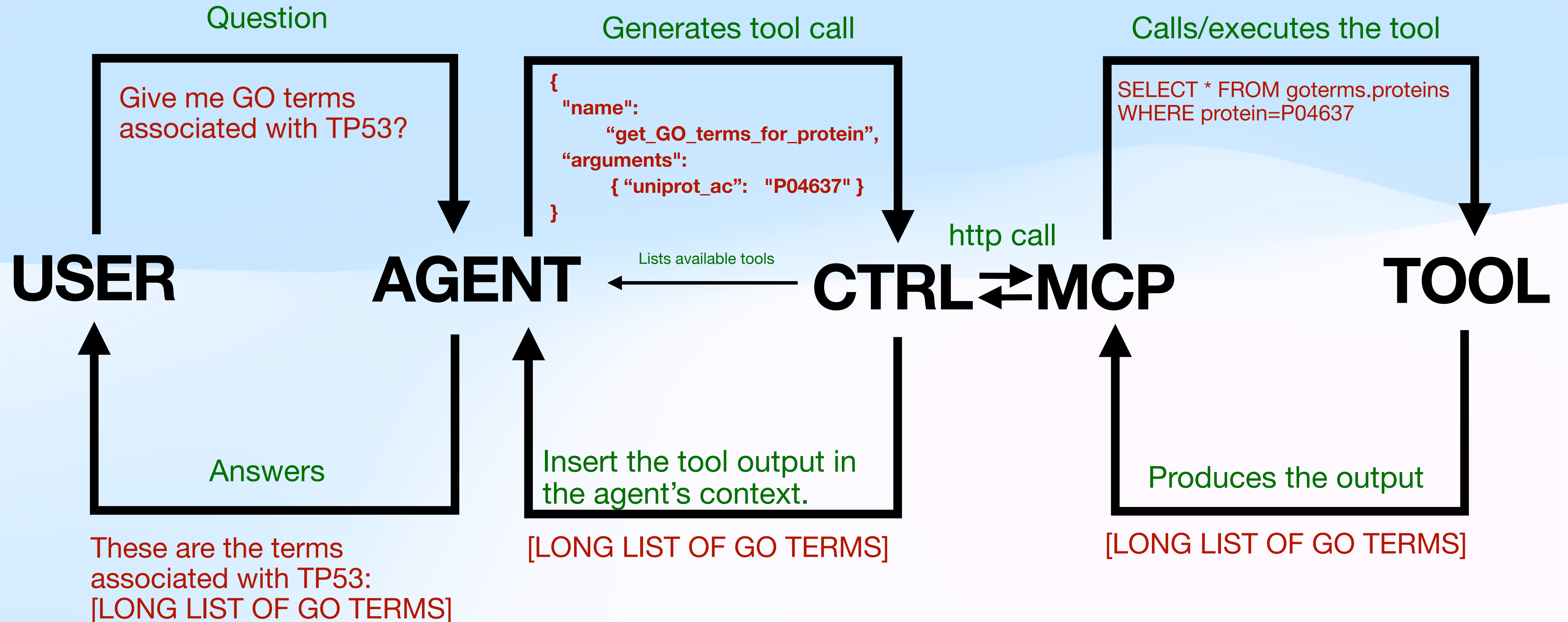
Good stuff:

- Simple to implement and integrate
- MCP can add metadata, context, or follow-up steps for the agent
- Works with existing infrastructure and APIs:
 - Does not require owning the underlying data source
 - Most APIs are already optimized for speed
- Can combine API calls with code (e.g. statistical analysis)

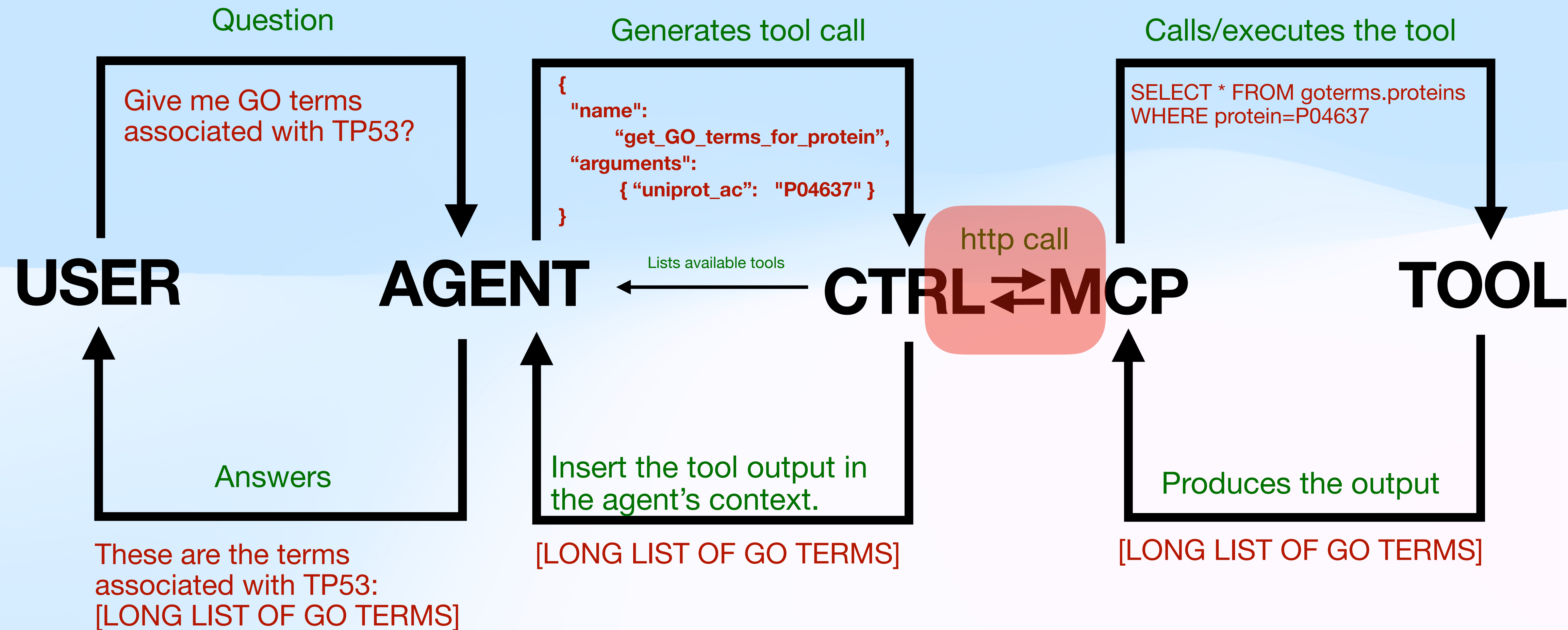
PART 2

The Protocol

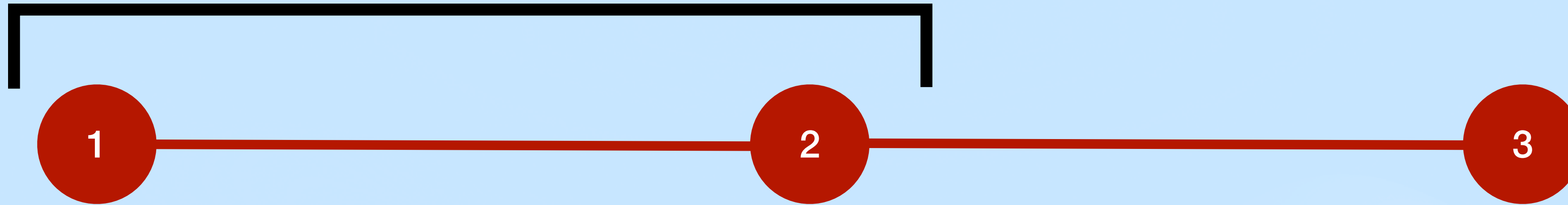
Agent-MCP workflow...



Agent-MCP workflow...



Init phase



Negotiate

Protocol version

Capabilities

Identity

Discover

Name

Descriptions

Input schema

Call

Name + arguments

Results

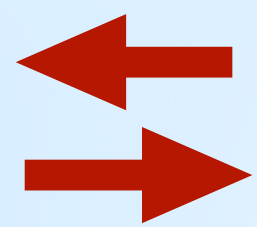
Output status

Initialization

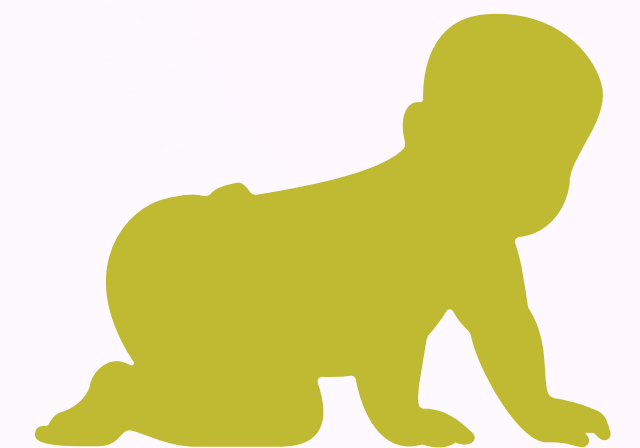
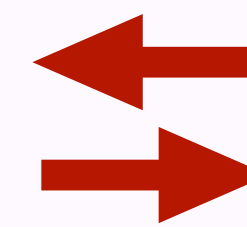
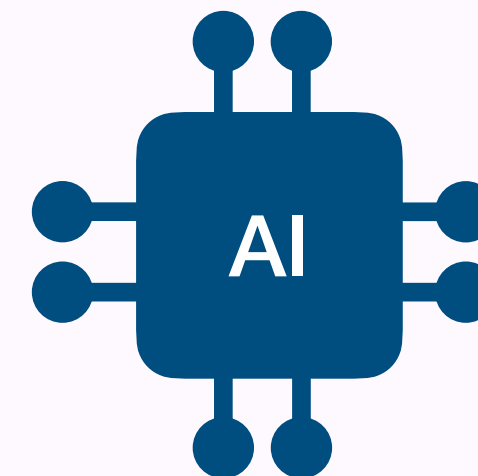
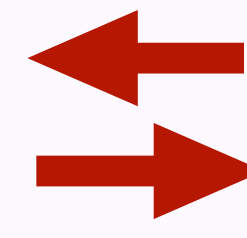
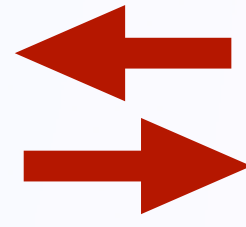
```
from mcp.server import MCPServer

mcp = MCPServer("string-workshop")

mcp.run(
    transport="streamable-http",
    host="0.0.0.0",
    port=8000,
    stateless_http=True,
)
```



NGINX

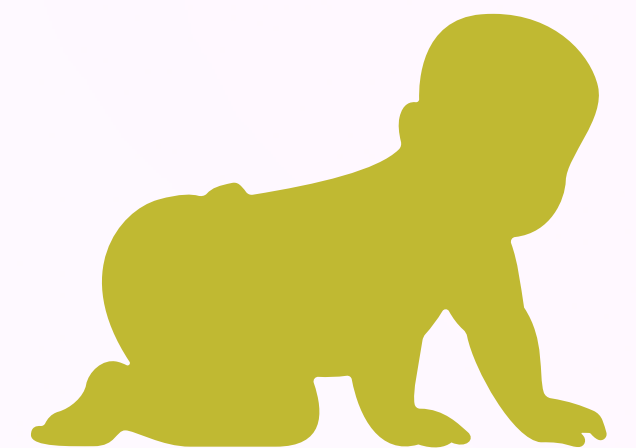
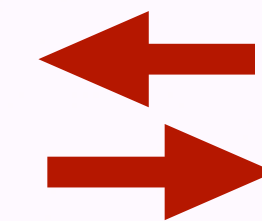
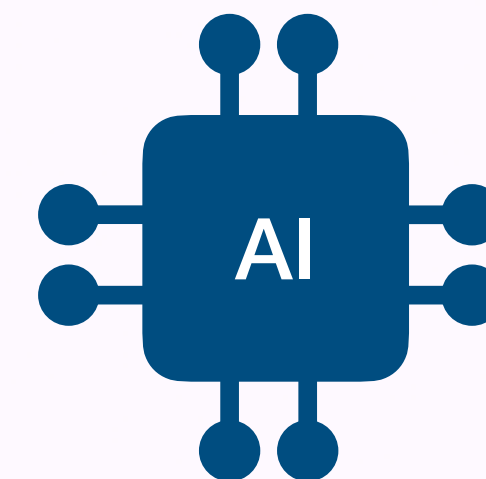


Initialization

```
from mcp.server import MCPServer

mcp = MCPServer("string-workshop")

mcp.run(
    transport="streamable-http",
    host="0.0.0.0",
    port=8000,
    stateless_http=True,
)
```

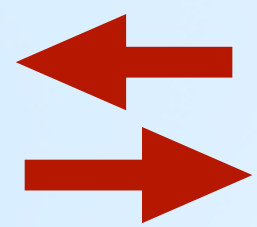


Initialization

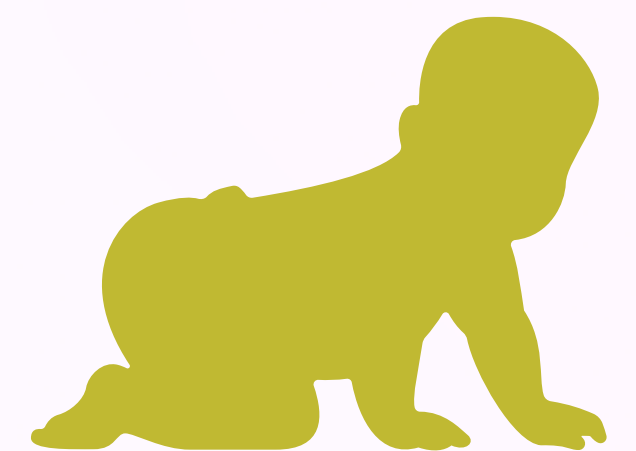
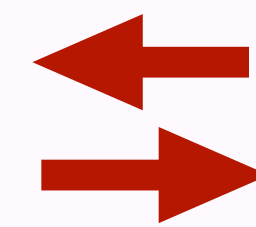
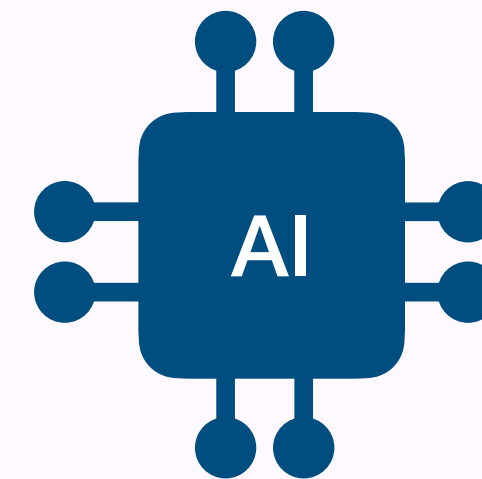
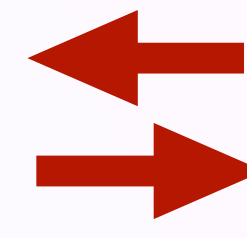
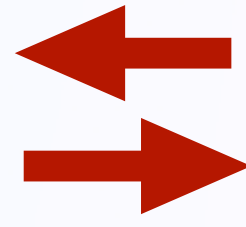
```
from mcp.server import MCPServer

mcp = MCPServer("string-workshop")

mcp.run(
    transport="streamable-http",
    host="0.0.0.0",
    port=8000,
    stateless_http=True,
)
```



NGINX

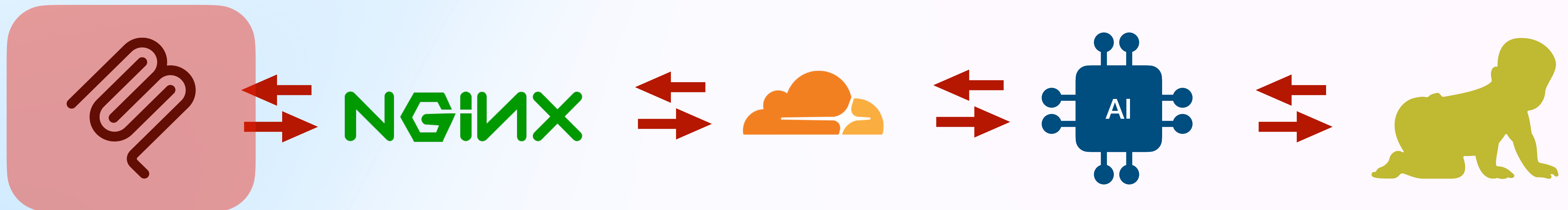


Initialization (MCP side)

```
from mcp.server import MCPServer

mcp = MCPServer("string-workshop")

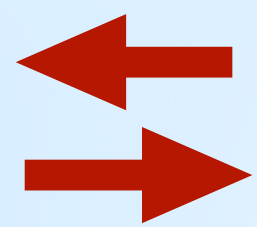
mcp.run(
    transport="streamable-http",
    host="0.0.0.0",
    port=8000,
    stateless_http=True,
)
```



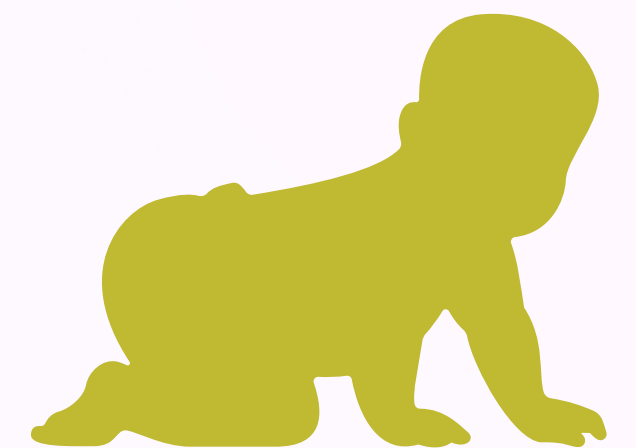
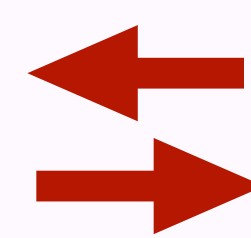
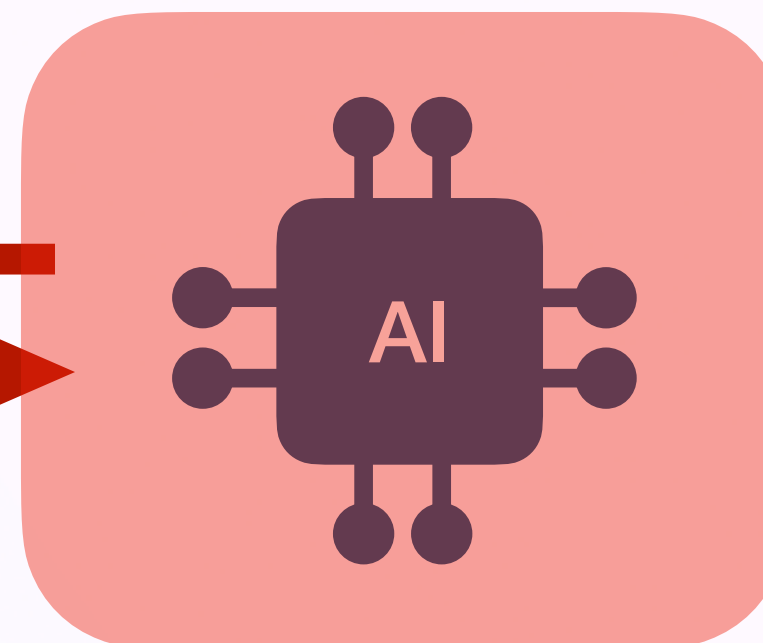
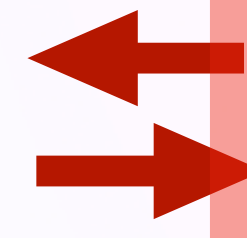
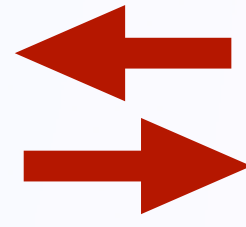
Initialization (agent side)

```
client = OpenAI(api_key=config["OPENAI_API_KEY"])

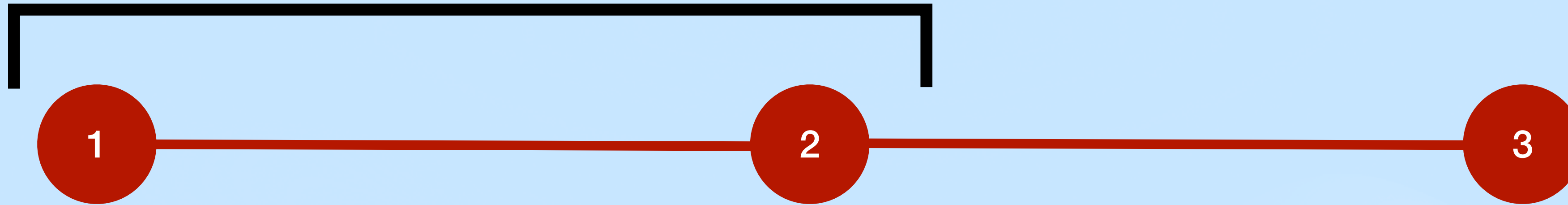
response = client.responses.create(
    model="gpt-5-nano",
    instructions=system_prompt,
    input=user_input,
    previous_response_id=previous_response_id,
    tools=[
        {
            "type": "mcp",
            "server_label": "mcp-workshop",
            "server_url": https://mcp.string-db.org/,
            "require_approval": "never",
        }
    ],
)
```



NGINX



Init phase



Negotiate

Protocol version
Capabilities
Identity

Discover

Name
Descriptions
Input schema

Call

Name + arguments
Results
Output status

Tool schema

```
@mcp.tool(title="Tool name")
def tools_function_name(
    parameter_name: Annotated[
        str,
        Field(
            description=(
                "Parameter description"
            )
        ),
    ],
) -> dict:
    """
    Tool description
    """
    Tool code

    return output_dictionary
```


Tool schema

```
@mcp.tool(title="Math toolbox: add two numbers")
def tool_box_add_two_numbers(
    first_number: Annotated[
        int,
        Field(
            description=(
                "First number"
            )
        ),
    ],
    second_number: Annotated[
        int,
        Field(
            description=(
                "Second number"
            )
        ),
    ]
) -> dict:
    """
    This tools adds to number and returns its sum
    """

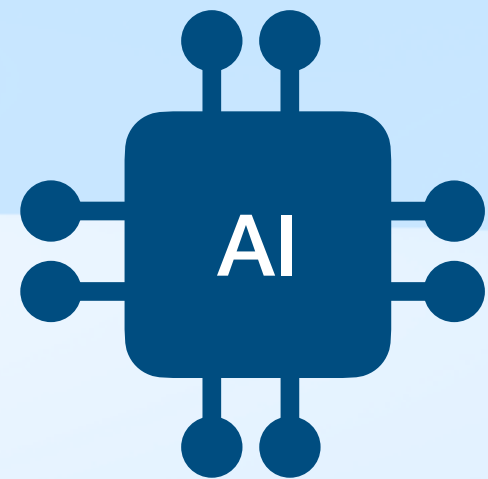
    sum = first_number + second_number
    return {"sum": sum}
```

Tool schema

```
@mcp.tool(title="STRING: Search human Biological Process terms")
def string_earch_human_biological_process_terms(
    query: Annotated[
        str,
        Field(
            description=(
                "Required. Text to search for in human Gene Ontology Biological
                Process term descriptions. Searches case-insensitively. Example: cell cycle"
            )
        ),
    ],
    limit: Annotated[
        Optional[int],
        Field(
            description="Optional. Maximum number of matching rows to return. Default: 5.",
            ge=1,
            le=20,
        ),
    ] = 5,
) -> dict:
    """
    Finds human Gene Ontology Biological Process terms whose descriptions match
    the supplied text.

    Each match contains a GO ID, and term description.
    """
```

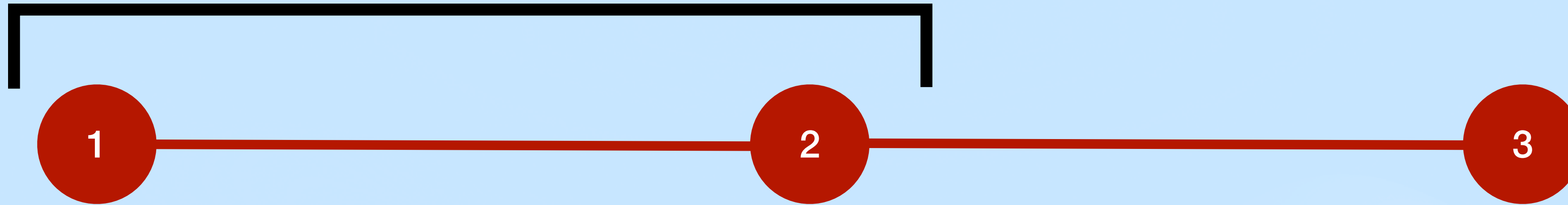
Discovery phase



```
@mcp.tool(title="STRING: Search human Biological Process terms")
def search_human_biological_process_terms(
    query: Annotated[
        str,
        Field(
            description=(
                "Required. Text to search for in human Gene Ontology Biological
                Process term descriptions. Searches case-insensitively. Example: cell cycle"
            )
        ),
    ],
    limit: Annotated[
        Optional[int],
        Field(
            description="Optional. Maximum number of matching rows to return. Default: 5.",
            ge=1,
            le=20,
        ),
    ] = 5,
) -> dict:
    """
    Finds human Gene Ontology Biological Process terms whose descriptions match
    the supplied text.

    Each match contains a STRING protein ID, GO ID, and term description.
    """
```


Init phase



Negotiate

Protocol version
Capabilities
Identity

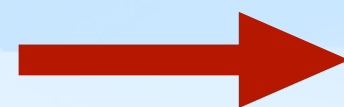
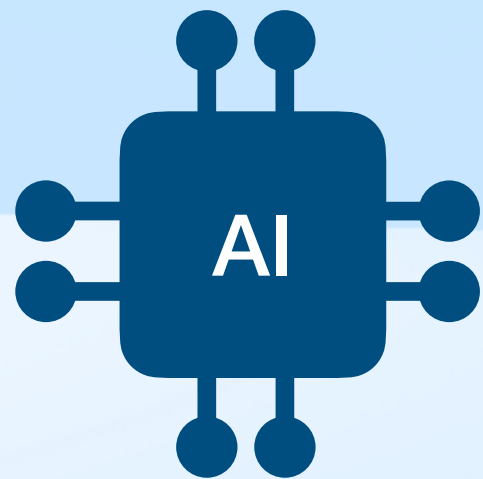
Discover

Name
Descriptions
Input schema

Call

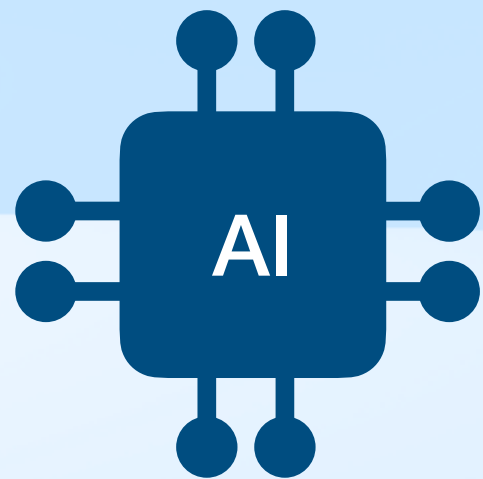
Name + arguments
Results
Output status

Call phase



```
{  
  "jsonrpc": "2.0",  
  "id": 7,  
  "method": "tools/call",  
  "params": {  
    "name": "search_human_biological_process_terms",  
    "arguments": {  
      "query": "cell",  
      "limit": 5  
    }  
  }  
}
```

Call phase



```
search_text = query.strip()

matches: set[tuple[str, str]] = set()
more_matches = False

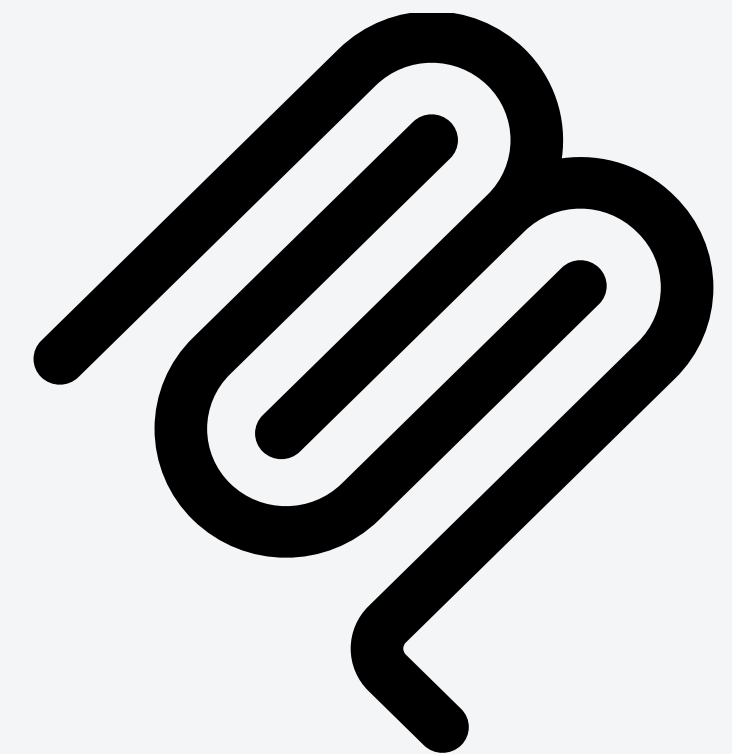
with open(HUMAN_BIOLOGICAL_PROCESS_TERMS, encoding="utf-8") as term_file:
    for line in term_file:
        _, _, go_id, description = line.rstrip().split("\t", maxsplit=3)
        if search_text.casefold() not in description.casefold():
            continue

        match = (go_id, description)
        if match in matches:
            continue

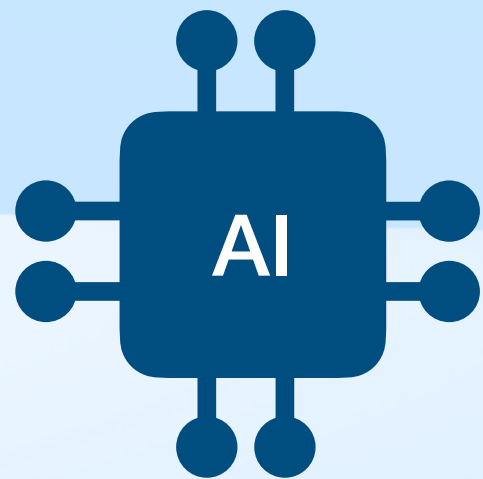
        if len(matches) >= limit:
            more_matches = True
            break

        matches.add(match)

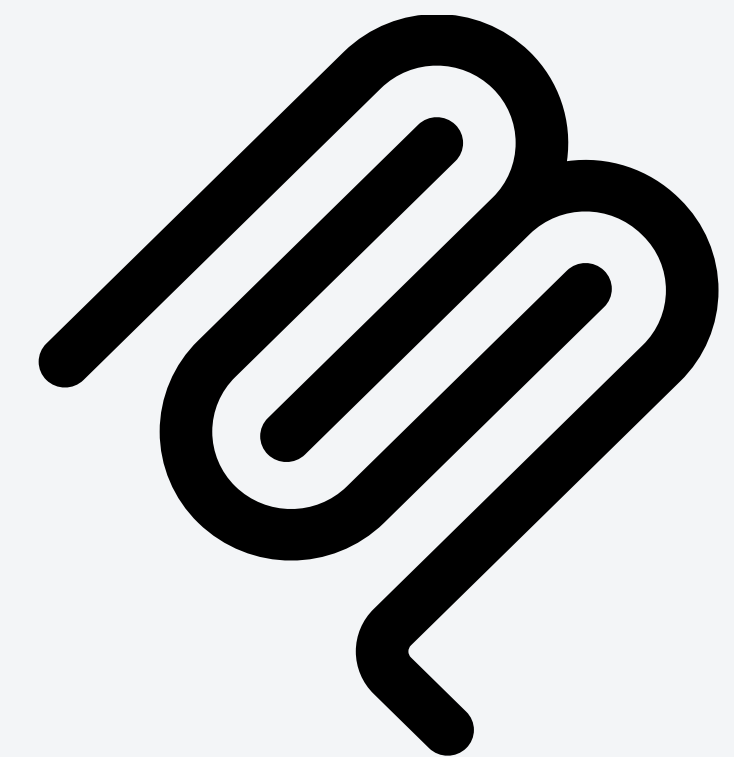
return {
    "query": search_text,
    "species": 9606,
    "matches": [
        {"go_id": go_id, "description": description}
        for go_id, description in sorted(matches)
    ],
    "more_matches": more_matches,
}
```



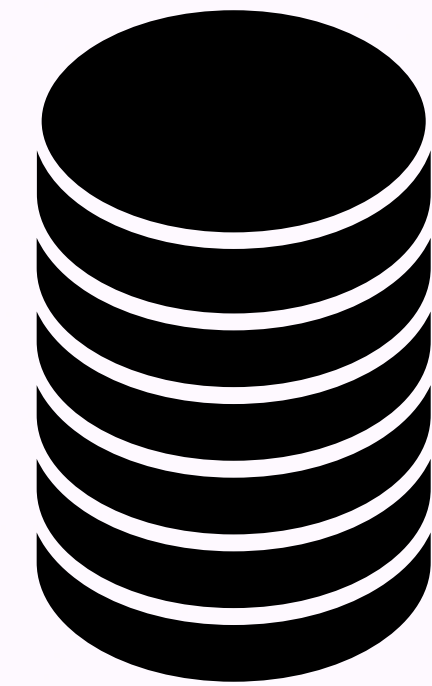
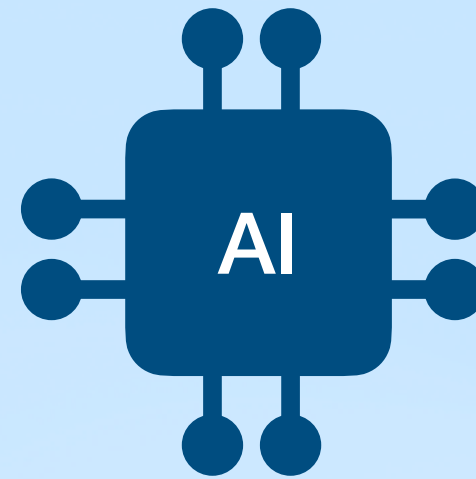
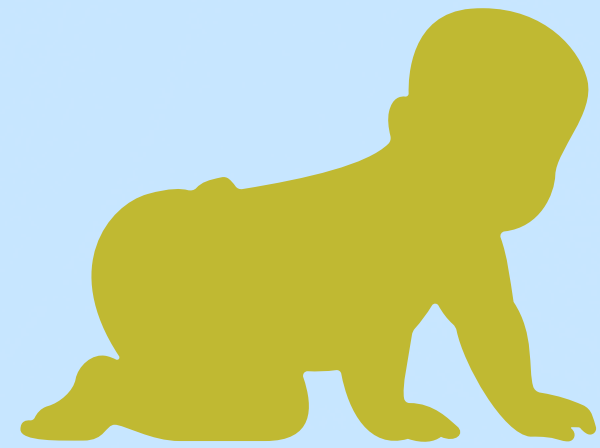
Call phase



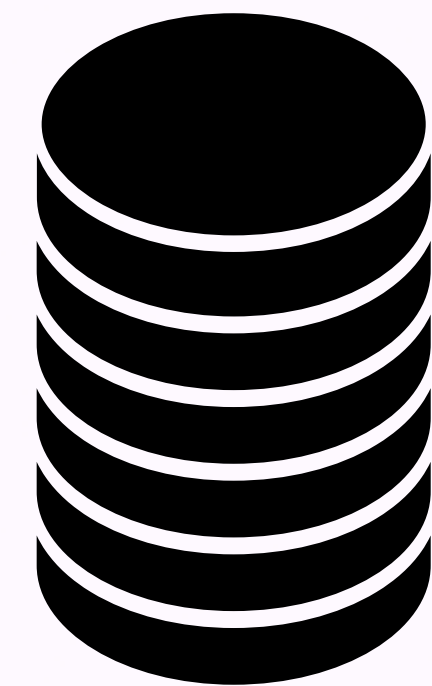
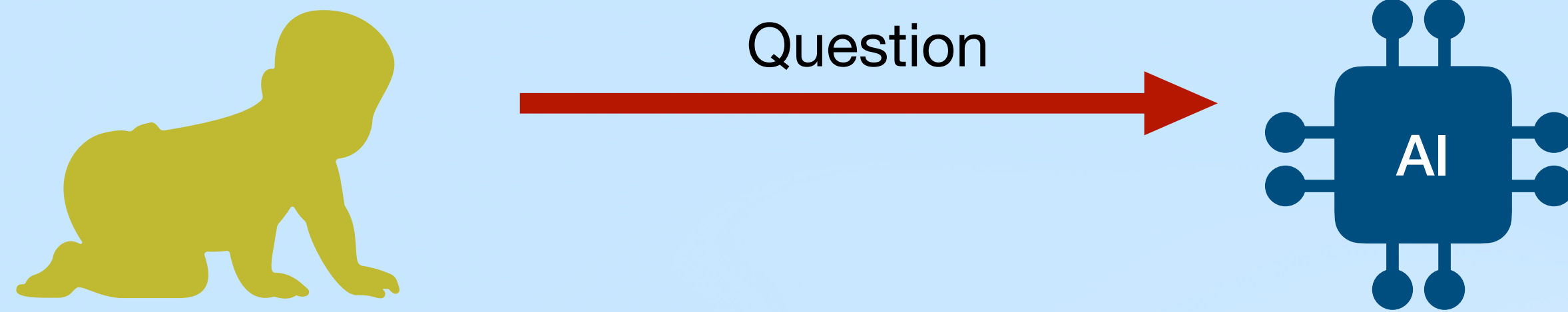
```
{
  "structuredContent": {
    "query": "cell",
    "species": 9606,
    "matches": [
      {
        "go_id": "GO:0007049",
        "description": "cell cycle"
      },
      {
        "go_id": "GO:0007155",
        "description": "cell adhesion"
      }
    ],
    "more_matches": true
  }
}
```



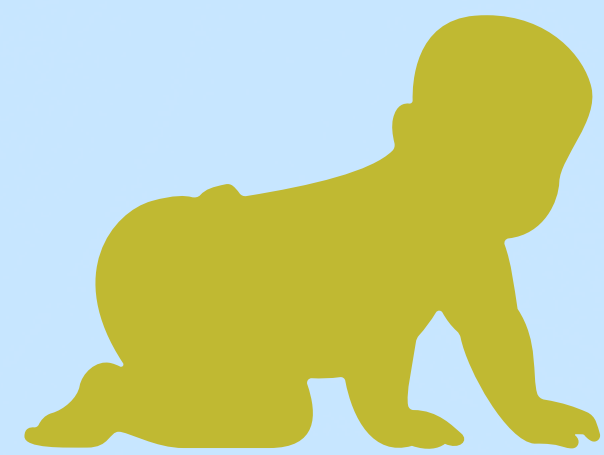
Workflow



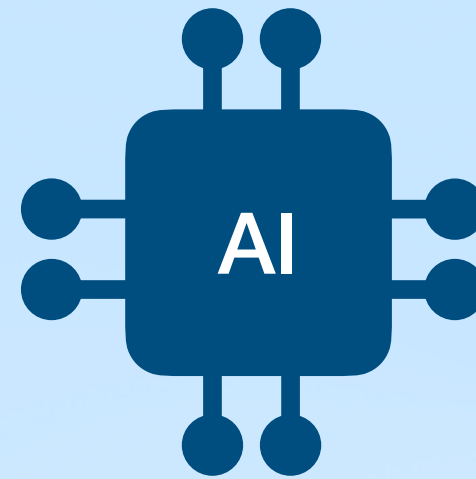
Workflow



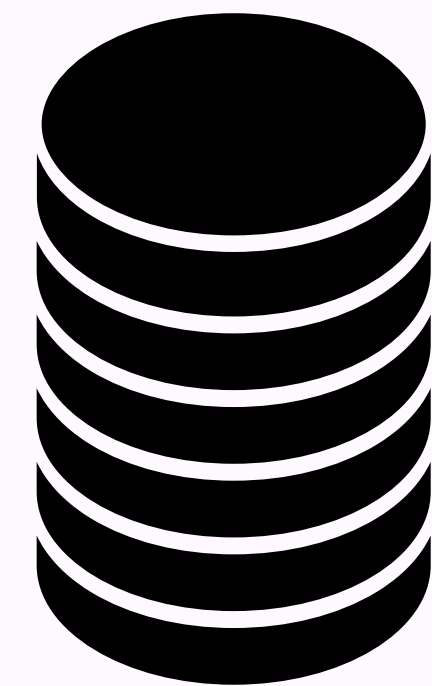
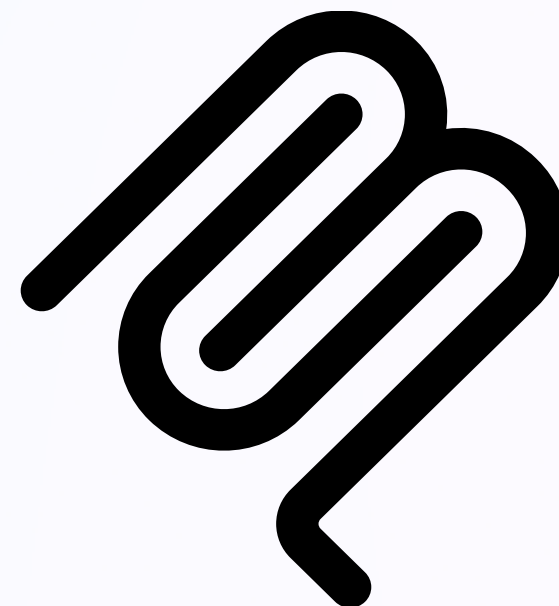
Workflow



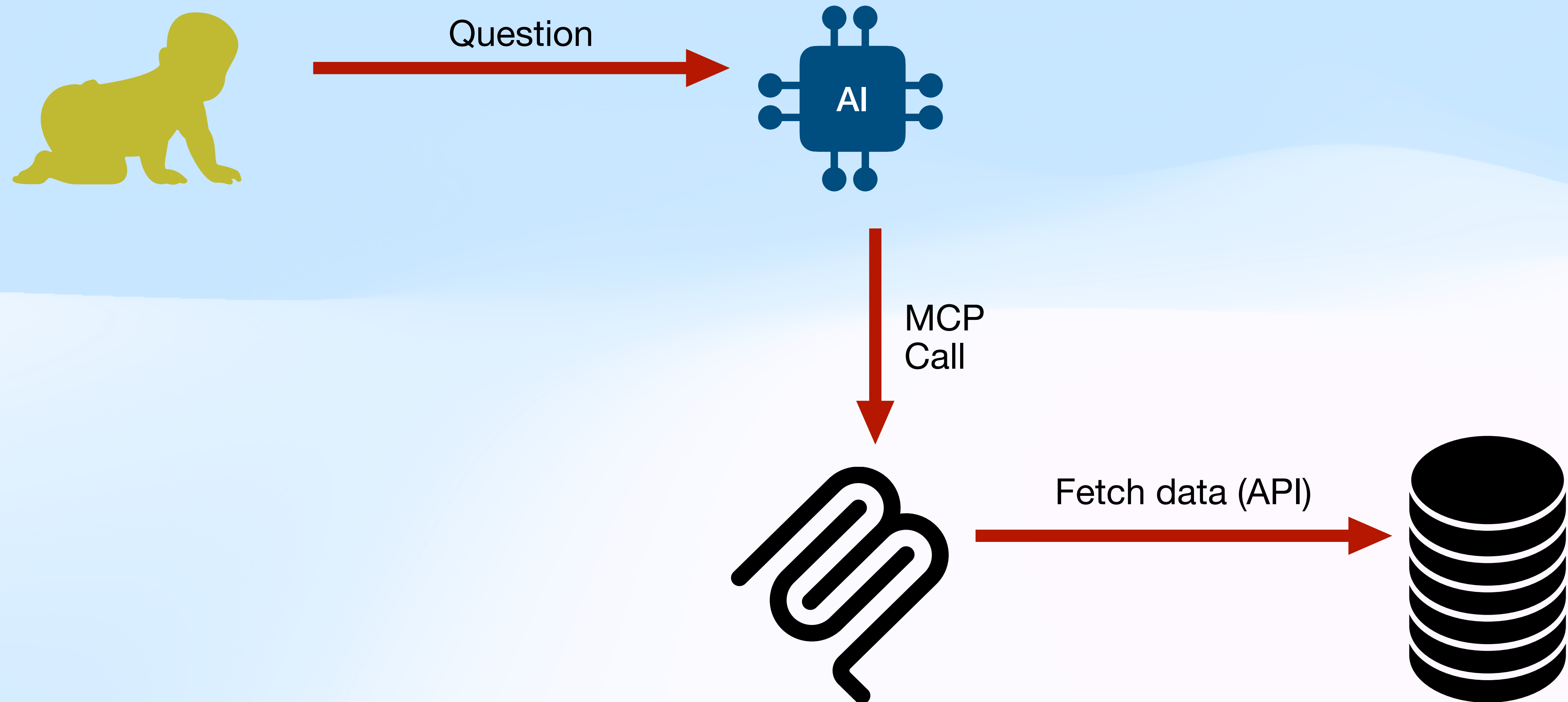
Question



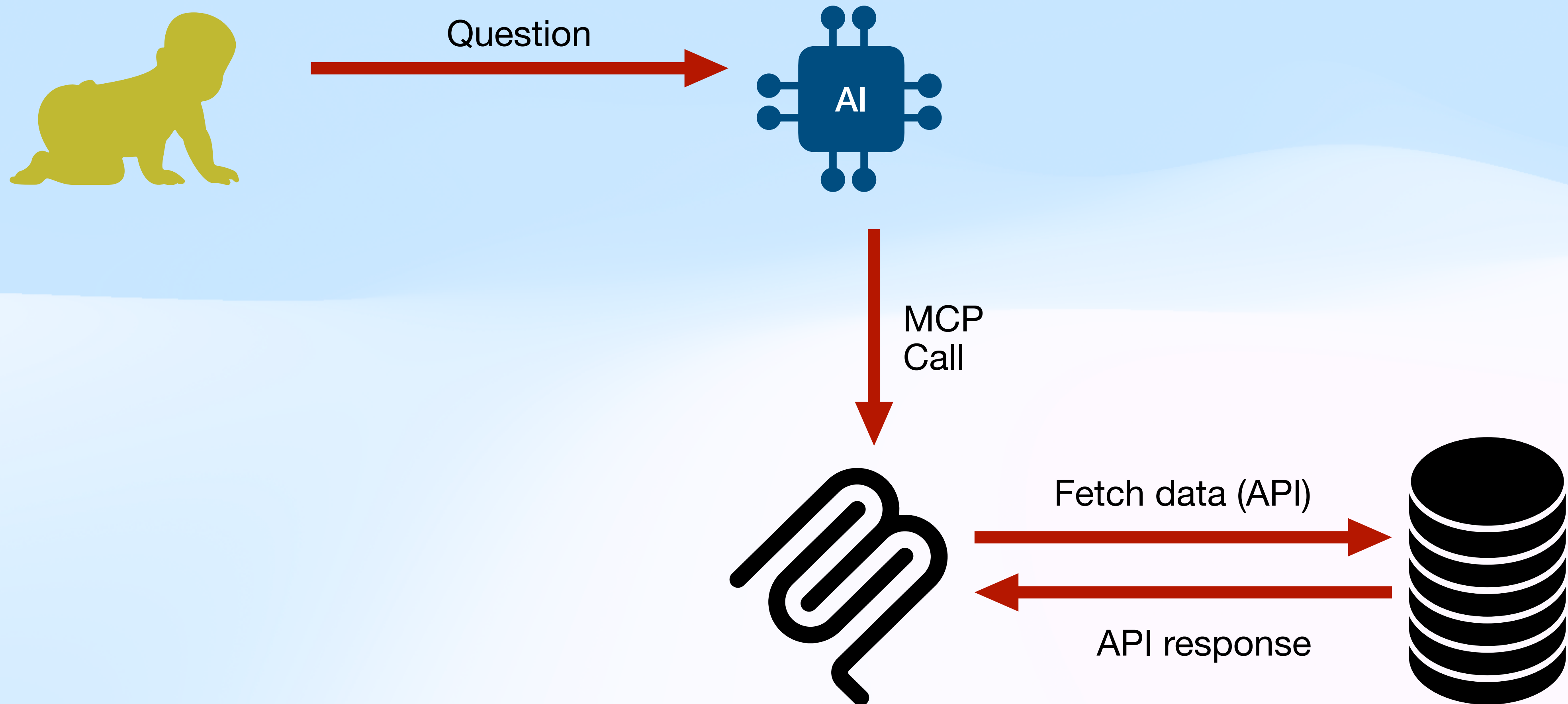
MCP
Call



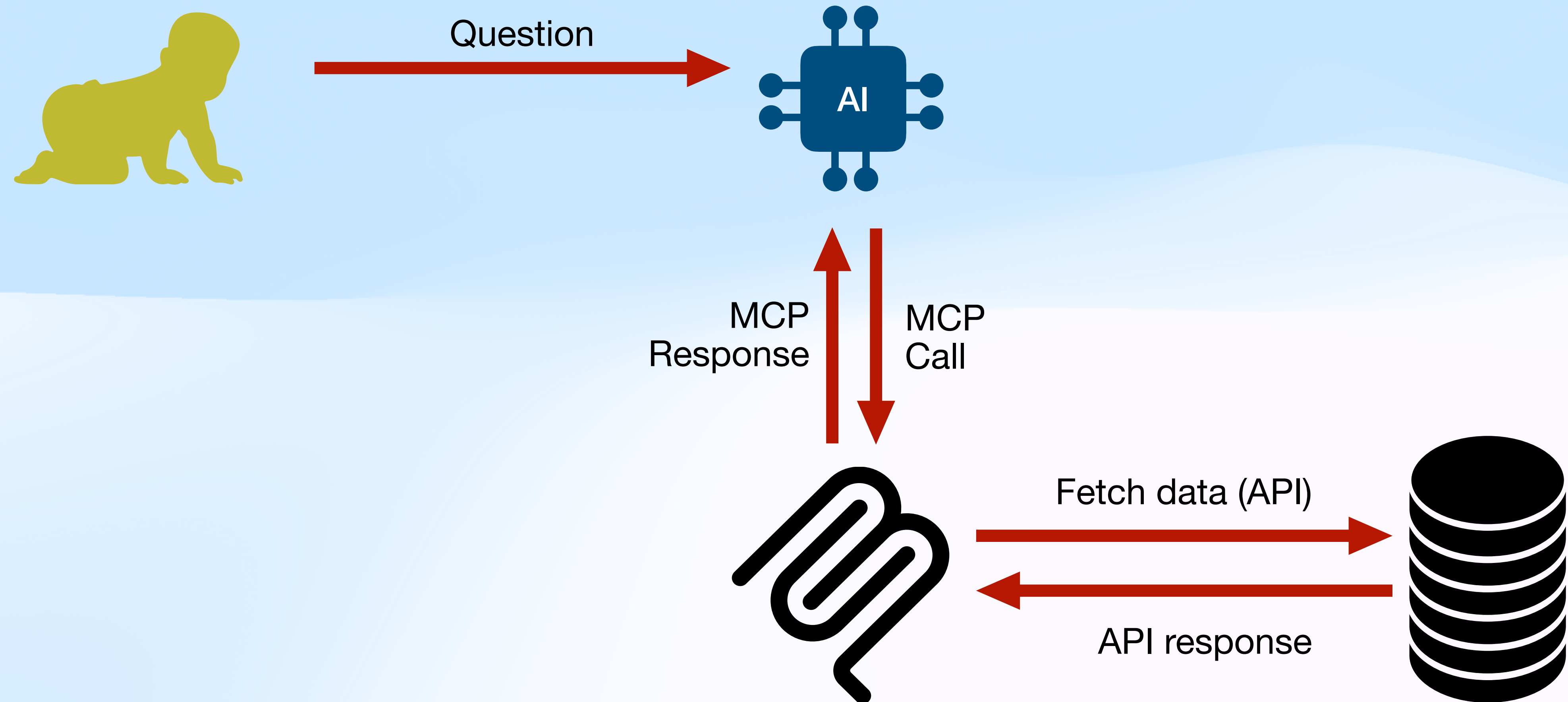
Workflow



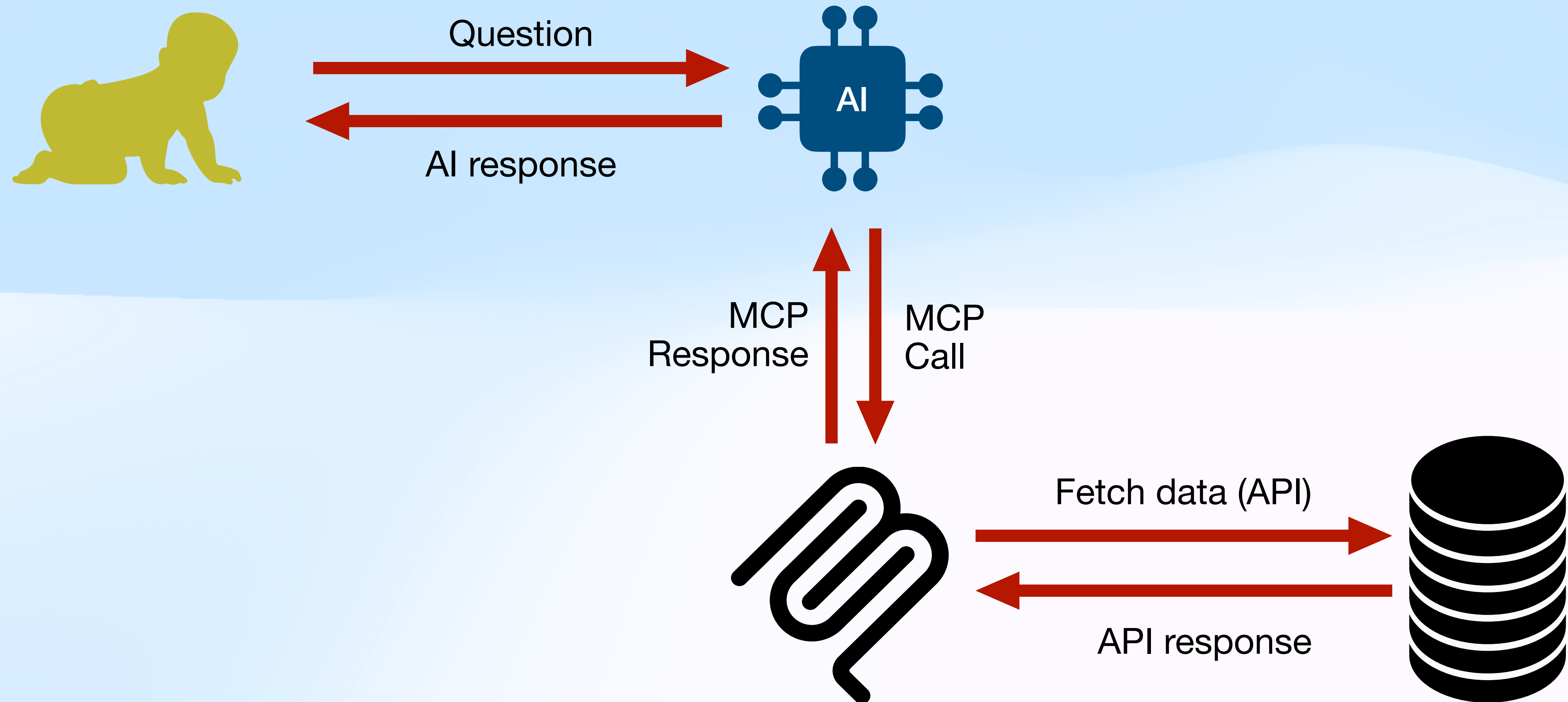
Workflow



Workflow



Workflow



PART 3

The tools

PART 3

Grounding

Grounding

Grounding means basing the model's answer on relevant external evidence supplied in its context.

Context

Context is everything the model can see to generate the answer.

- Instructions / System Prompt
- Tool names, descriptions, parameters
- History / conversation
- Current question
- Tool results

Context

Context is everything the model can see to generate the answer.

- Instructions / System Prompt
- Tool names, descriptions, parameters
- History / conversation
- Current question
- Tool results

Grounding

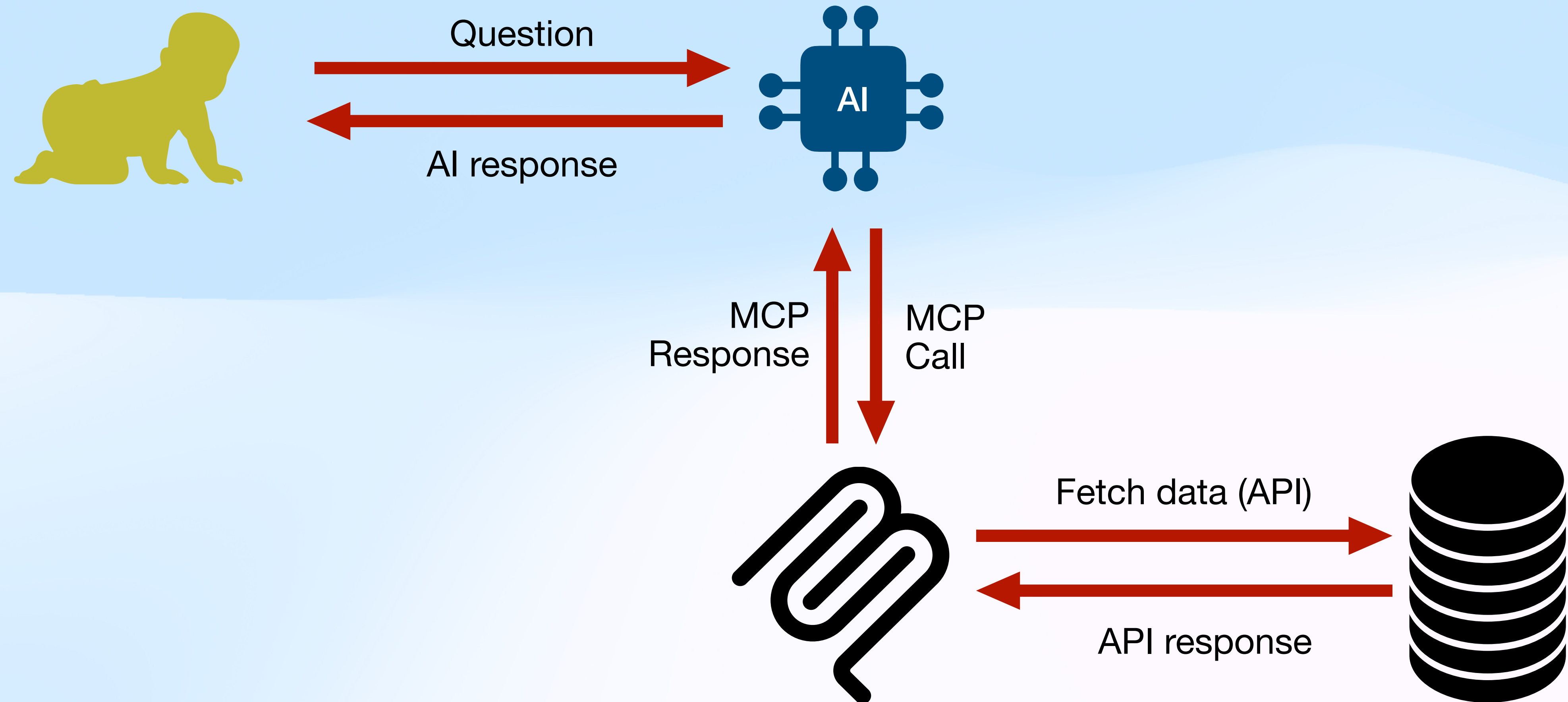
MCP explicit purpose is grounding...

- **Reducing pre-call context**
Fewer tools, concise descriptions and schemas
- **Reducing post-call context**
Relevant fields, filtered and limited results
- **Clarify parameters**
Reduce param space, defaults, examples
- **Clarify outputs**
Metadata, informative failures
- **Preserve the evidence trail**

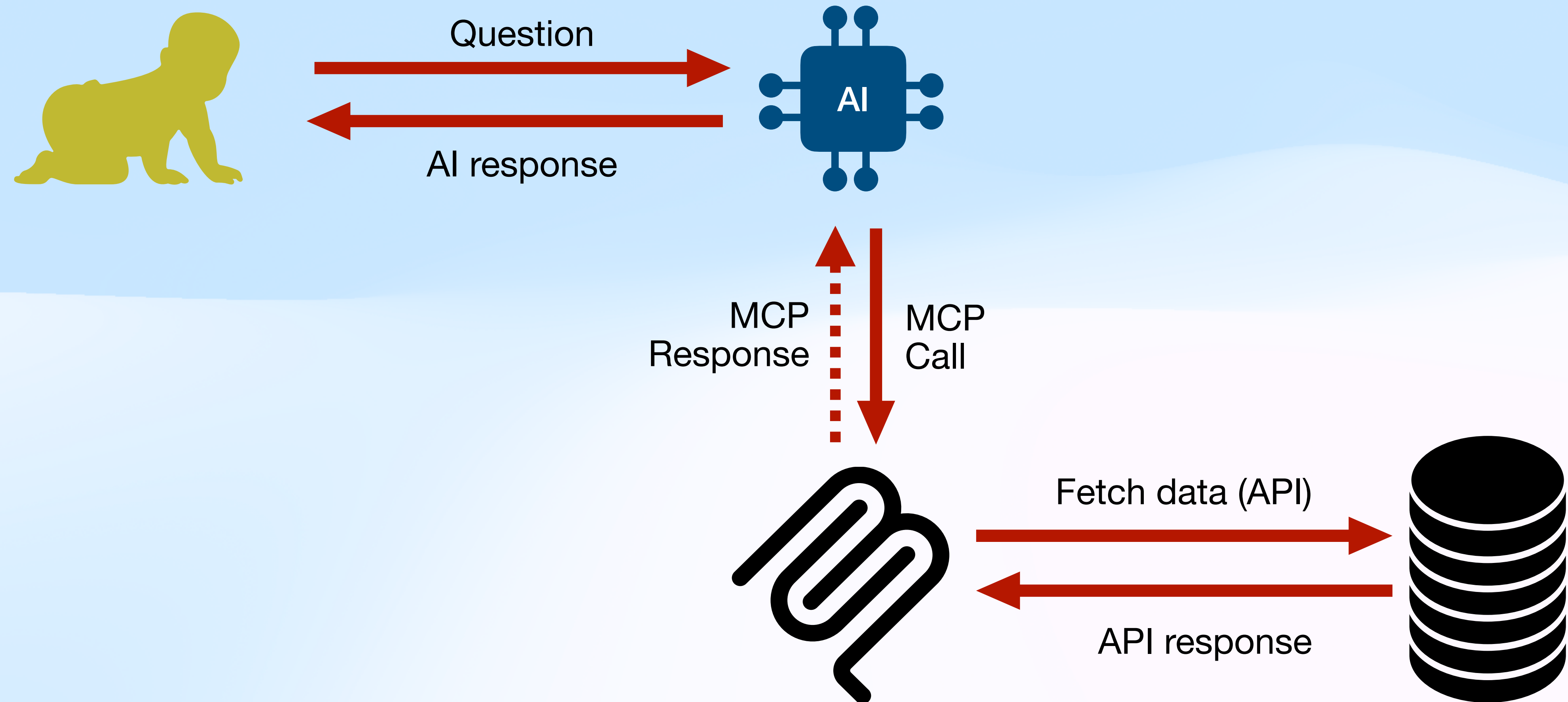
Traceability

Question → authoritative data → context → answer

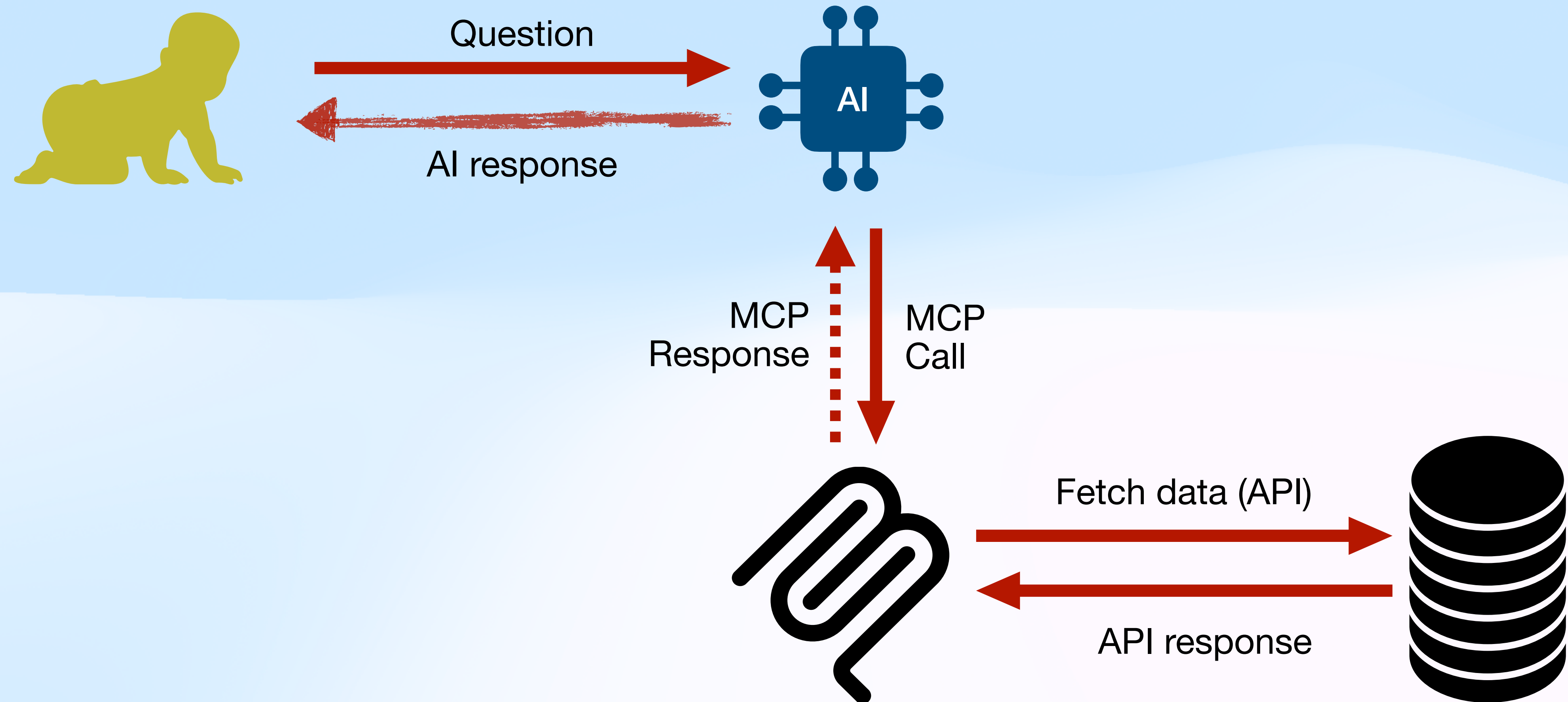
Workflow (Traceability)



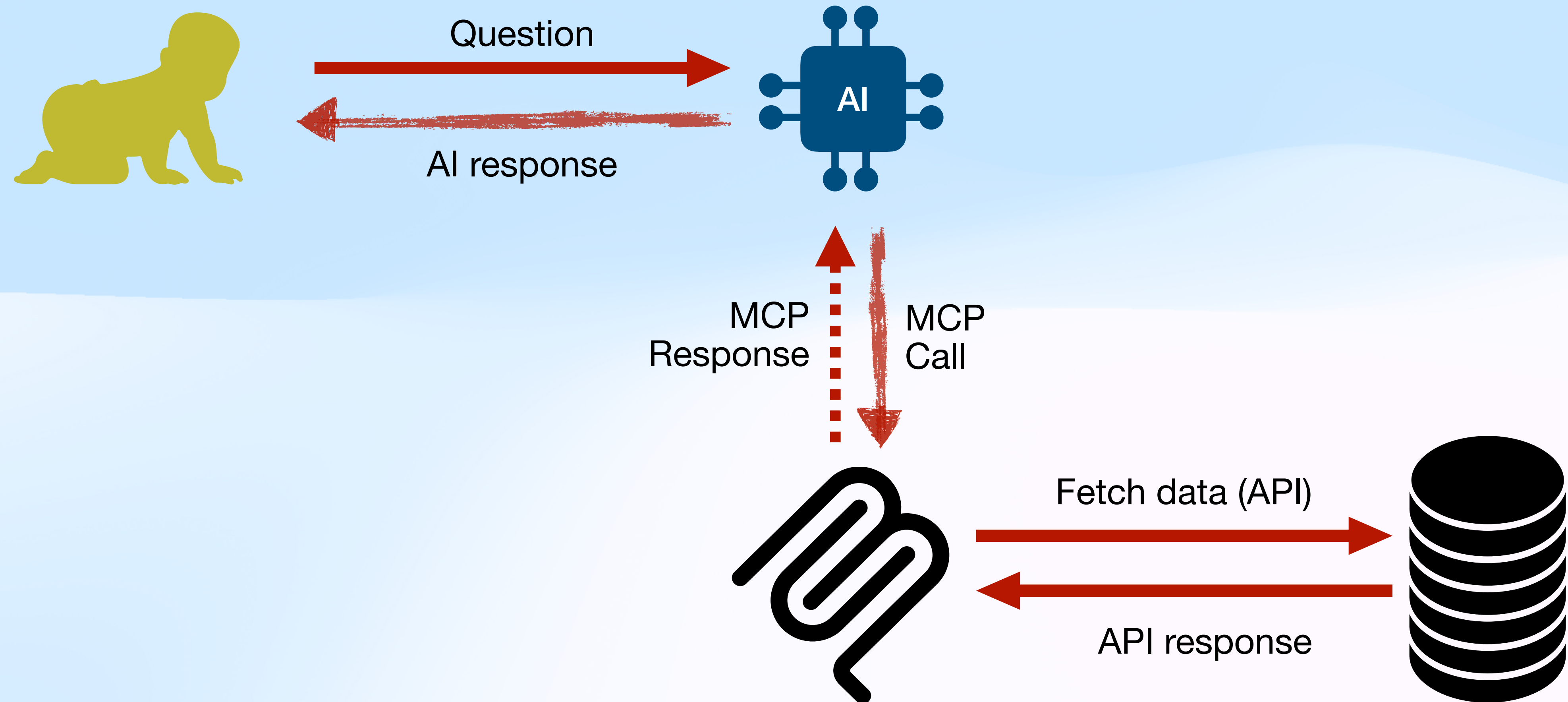
Workflow (Traceability)



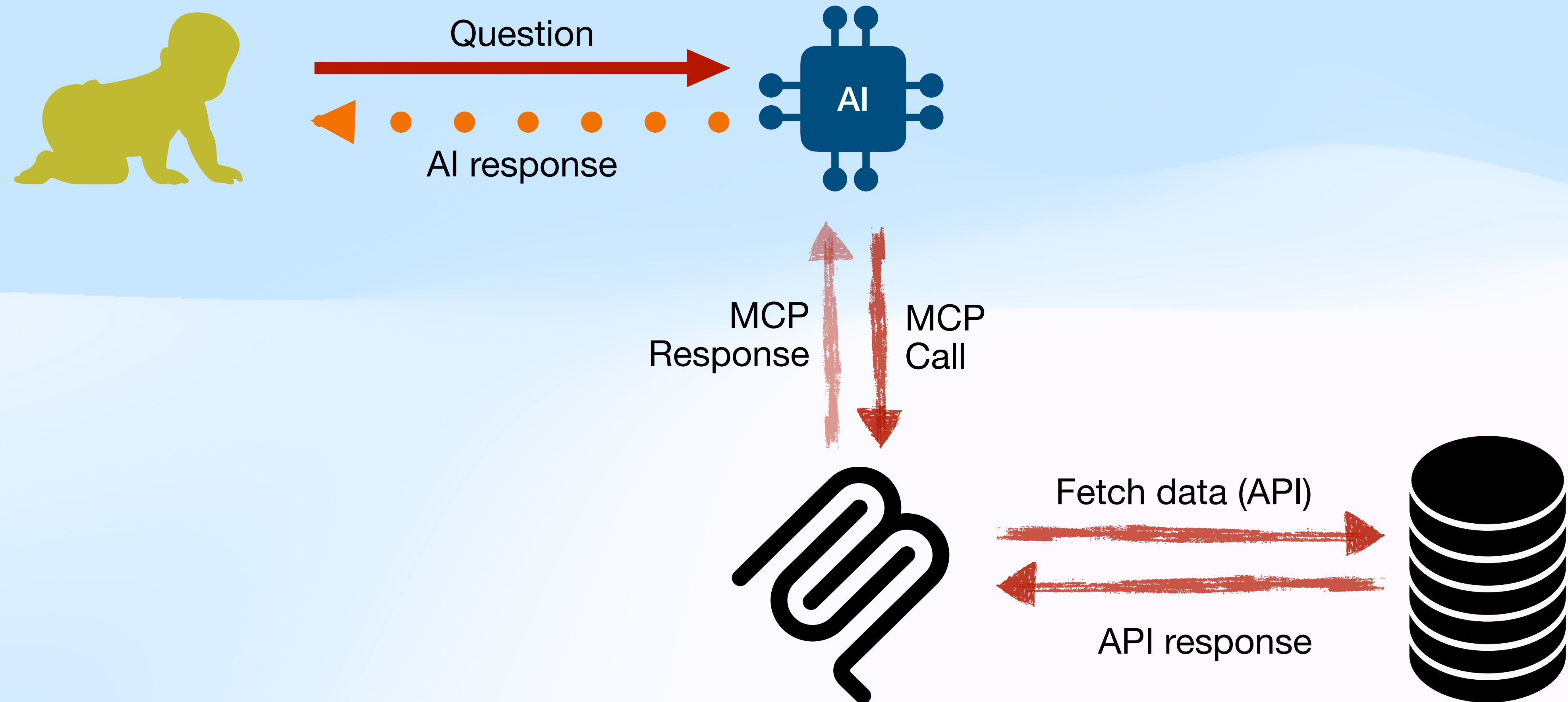
Workflow (Traceability)



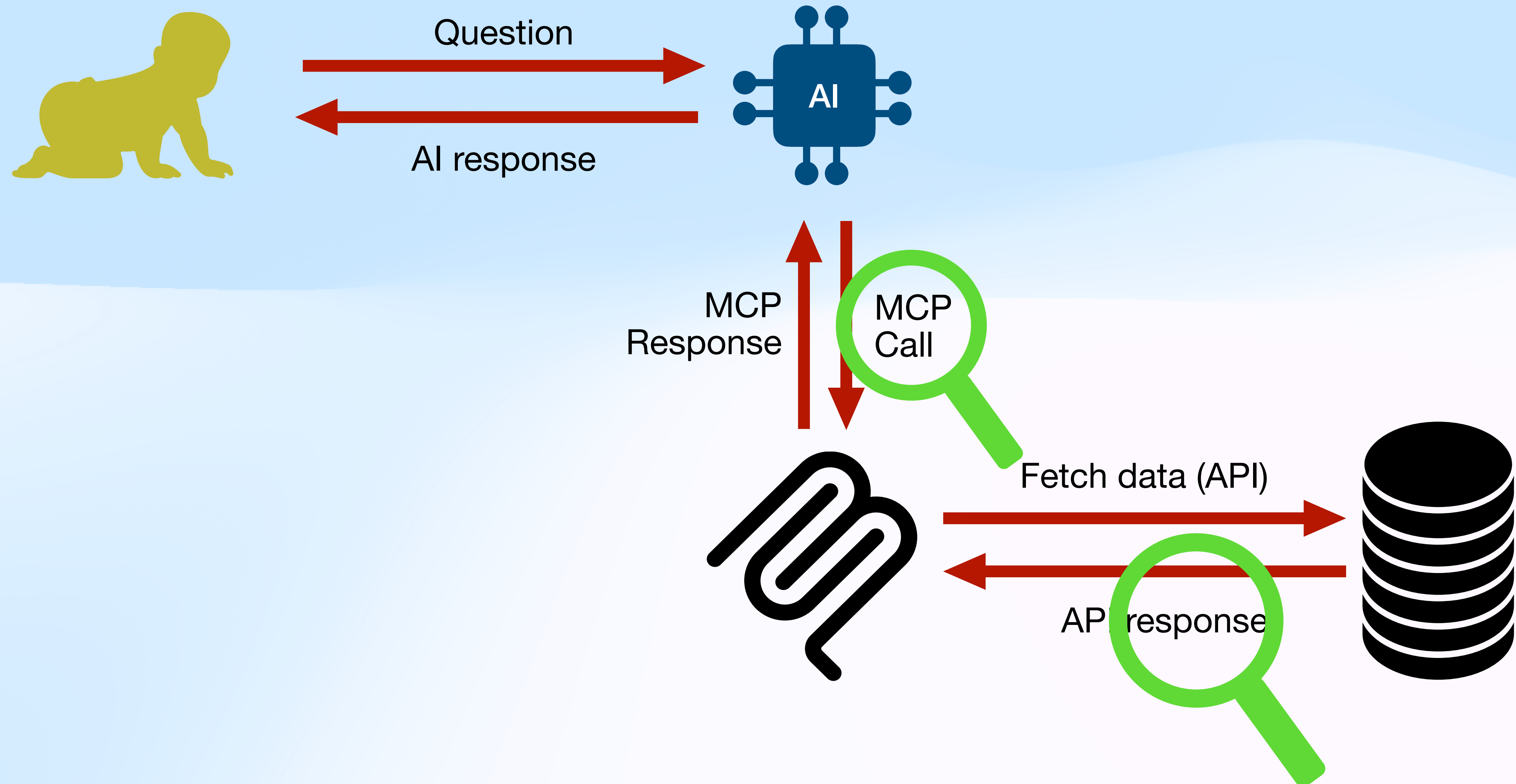
Workflow (Traceability)



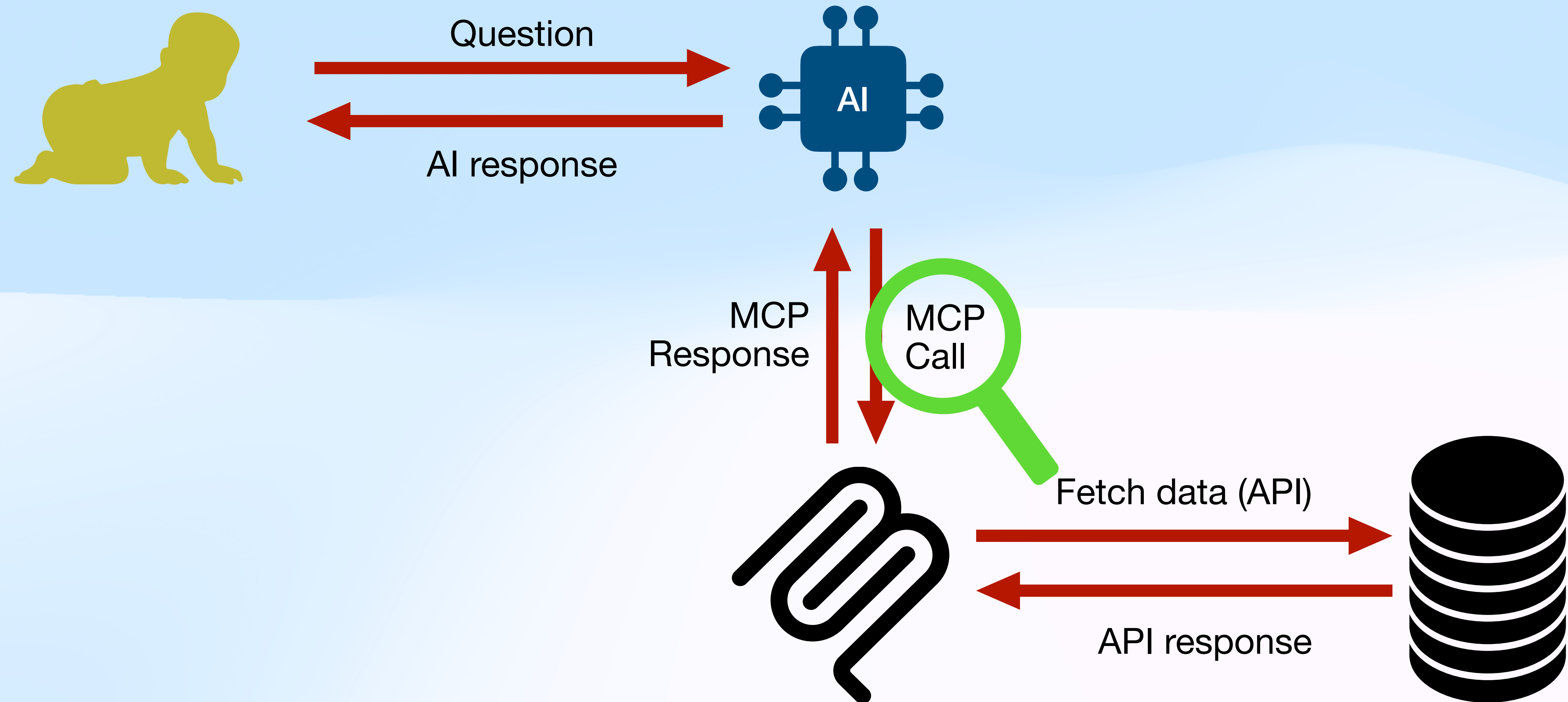
Workflow (Traceability)



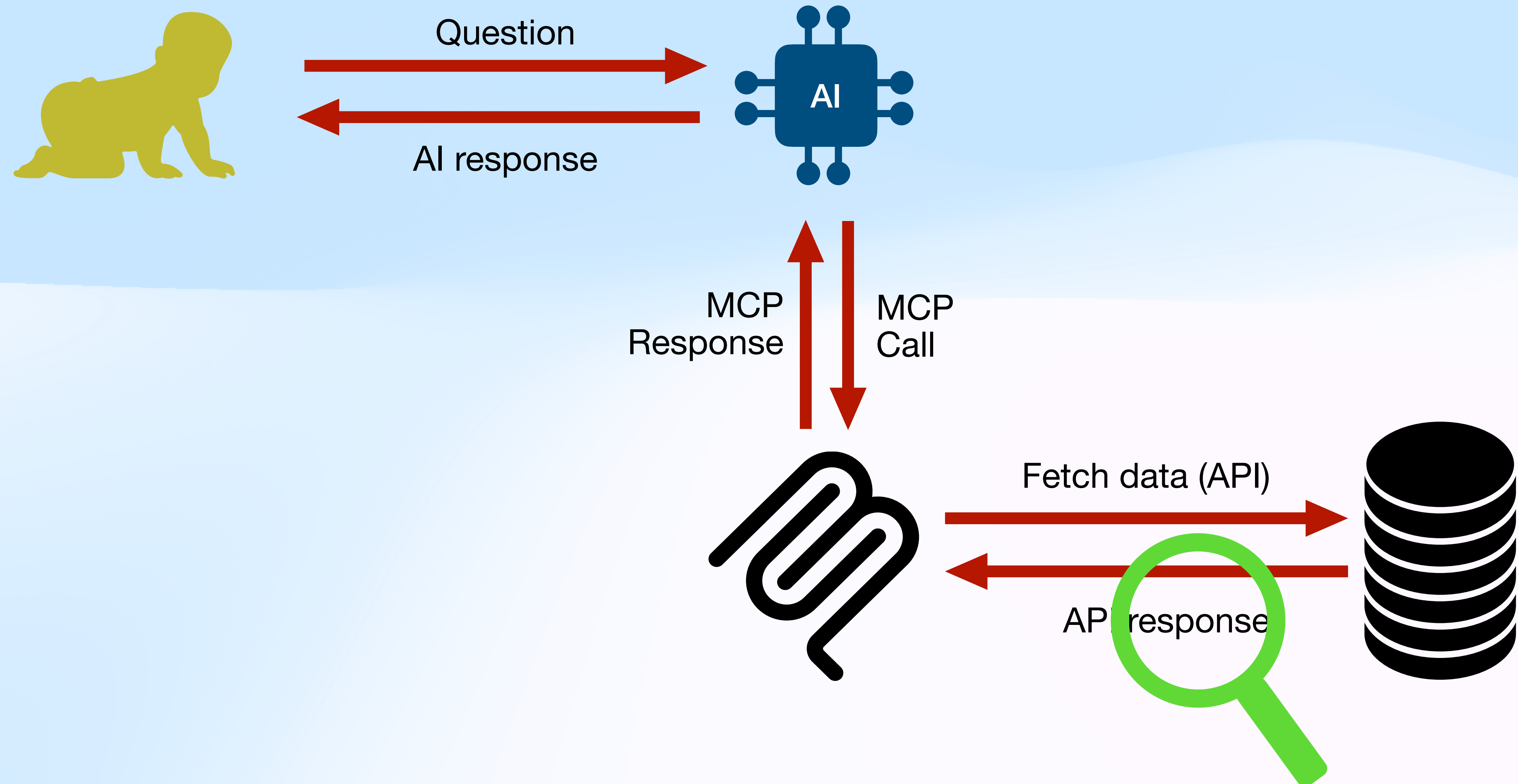
Workflow (Traceability)



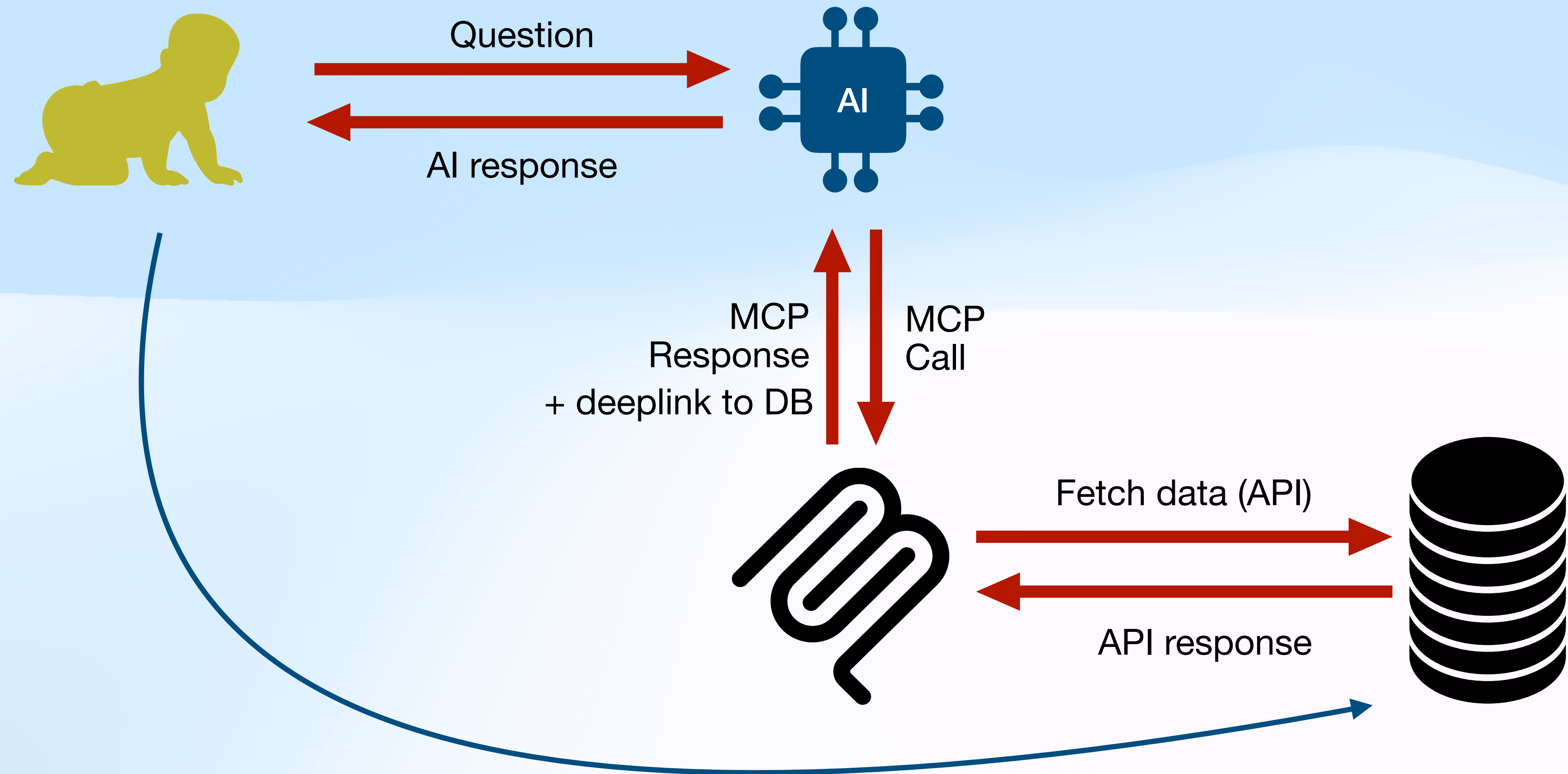
Workflow (Traceability)



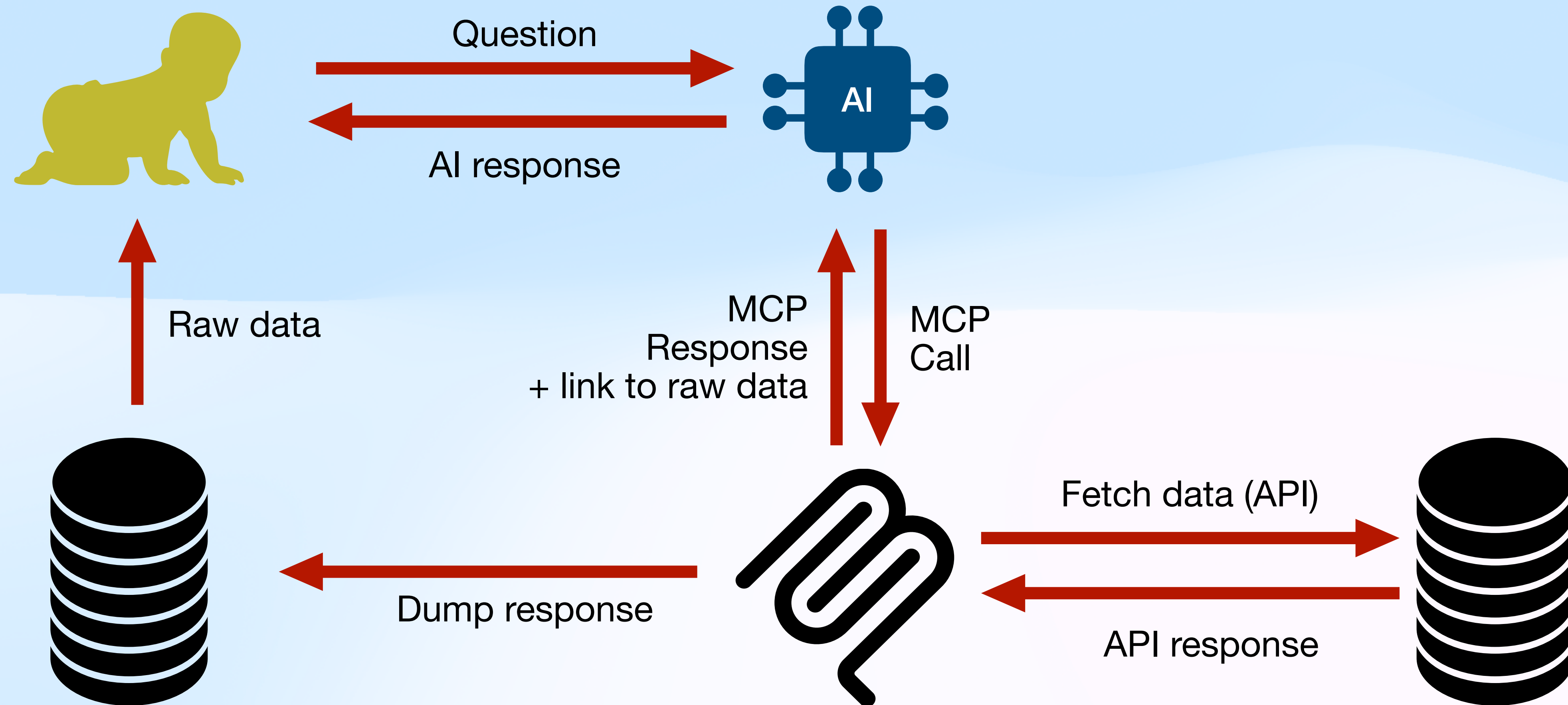
Workflow (Traceability)



Workflow (Traceability)



Workflow (Traceability)



PART 4

The tools

Tool names

- Rather short and specific
 - Use verb + scientific object
 - avoid ambiguity between the tools
 - Do not overload the tool
- Include the resource name to prevent ambiguity
- Avoid generic names: search, query, get_data
- Keep naming conventions consistent

Tool names

- Rather short and specific
 - Use verb + scientific object
 - avoid ambiguity between the tools
 - Do not overload the tool
- Include the resource name to prevent ambiguity
- Avoid generic names: search, query, get_data
- Keep naming conventions consistent

```
@mcp.tool(  
    title="STRING: Find interactions within a protein set"  
)  
def string_interactions_query_set(...):  
  
@mcp.tool(  
    title="STRING: Find interaction partners"  
)  
def string_all_interaction_partners(...):
```

Tool names

- Rather short and specific
 - Use verb + scientific object
 - avoid ambiguity between the tools
 - Do not overload the tool
- Include the resource name to prevent ambiguity
- Avoid generic names: search, query, get_data
- Keep naming conventions consistent

NOPE!

```
@mcp.tool(
    title="STRING: Get interactions"
)
def string_get_interactions(
    within: Annotated[
        Optional[bool],
        Field(
            description="If this parameter is set to true then return the"
                        "interactions that are only within the query set otherwise"
                        "the tool returns interactions between the query set proteins"
                        "and all other proteins (outside the query set)."
        )
    ],
    ...
] = False,
```

Number of tools

- **Think about “tasks” not APIs**
 - Do not translate every API into a tool by default
- Try to build the smallest set possible to complete the tasks
- **Every tool consumes the context:**
 - provides name + params + description
 - Competes for agent attention
 - May cause ambiguity and confusion
- **No universal limit... 100 too much, 50 probably also too much**
 - newer version of MCP may provide search options
 - Keep in mind that your MCP might combined with others

Number of tools

NOPE!

```
@mcp.tool(
    title="STRING: Retrieve GO annotations"
)
def string_get_go_annotations(
    proteins: list[str],
) -> dict:
    ...

@mcp.tool(
    title="STRING: Retrieve KEGG annotations"
)

def string_get_kegg_annotations(
    proteins: list[str],
) -> dict:
    ...

@mcp.tool(
    title="STRING: Retrieve Reactome annotations"
)
def string_get_reactome_annotations(
    proteins: list[str],
) -> dict:
    ...
```

Number of tools

```
@mcp.tool(  
    title="STRING: Retrieve functional annotations"  
)  
def string_functional_annotation(  
    proteins: list[str],  
    sources: Annotated[  
        Optional[list[Literal["go", "kegg", "reactome"]]],  
        Field(  
            description=(  
                "Annotation sources to include. "  
                "Omit to return all available sources."  
            )  
        ),  
    ] = None,  
)
```


Parameters

- Concise and descriptive
- Can be helpful for defining the tool usage/output
- Allow for multi-identifier queries
- Keep them consistent between the tools
- Agents tend to over-use available parameters:
 - Their use might be inconsistent
 - avoid parameters that are not critical and detrimental to the output
- set good defaults and examples
- discourage use by “use only when...”

```
parameter_name: Annotated[
    type,                # or Optional[TYPE]
    Field(
        description="Parameter description.
                     "Can be multi-line",
        examples=[EXAMPLE_1, EXAMPLE_2],
        ge=MINIMUM,      # or gt=
        le=MAXIMUM,      # or lt=
        min_length=MIN_LENGTH,
        max_length=MAX_LENGTH,
    ),
] = DEFAULT
```

Description

```
@mcp.tool(title="STRING: Search human Biological Process terms")
def search_human_biological_process_terms(
    query: Annotated[
        str,
        Field(
            description=(
                "Text to search for in human Gene Ontology Biological
            )
        ),
    ]
) -> dict:
    """
    Finds human Gene Ontology Biological Process terms whose descriptions match
    the supplied text.

    Each match contains a STRING protein ID, GO ID, and term description.
    """

    [CODE GOES HERE]
```

Description

- Immutable (cannot be changed in runtime)
- Treat name and parameters as part of the description
 - do not repeat them...
- Do not describe the output unless the output is not self explanatory
- You can describe output, if the output helps to define the tool task.
- Iterate a lot, until you find short version that completes the task.

Description

```
@mcp.tool(title="STRING: Search human Biological Process terms")
def search_human_biological_process_terms(
    query: Annotated[
        str,
        Field(
            description=(
                "Text to search for in human Gene Ontology Biological
            )
        ),
    ]
) -> dict:
    """
    Finds human Gene Ontology Biological Process terms whose descriptions match
    the supplied text.

    Each match contains a STRING GO ID, and term description.
    """

    [CODE GOES HERE]
```

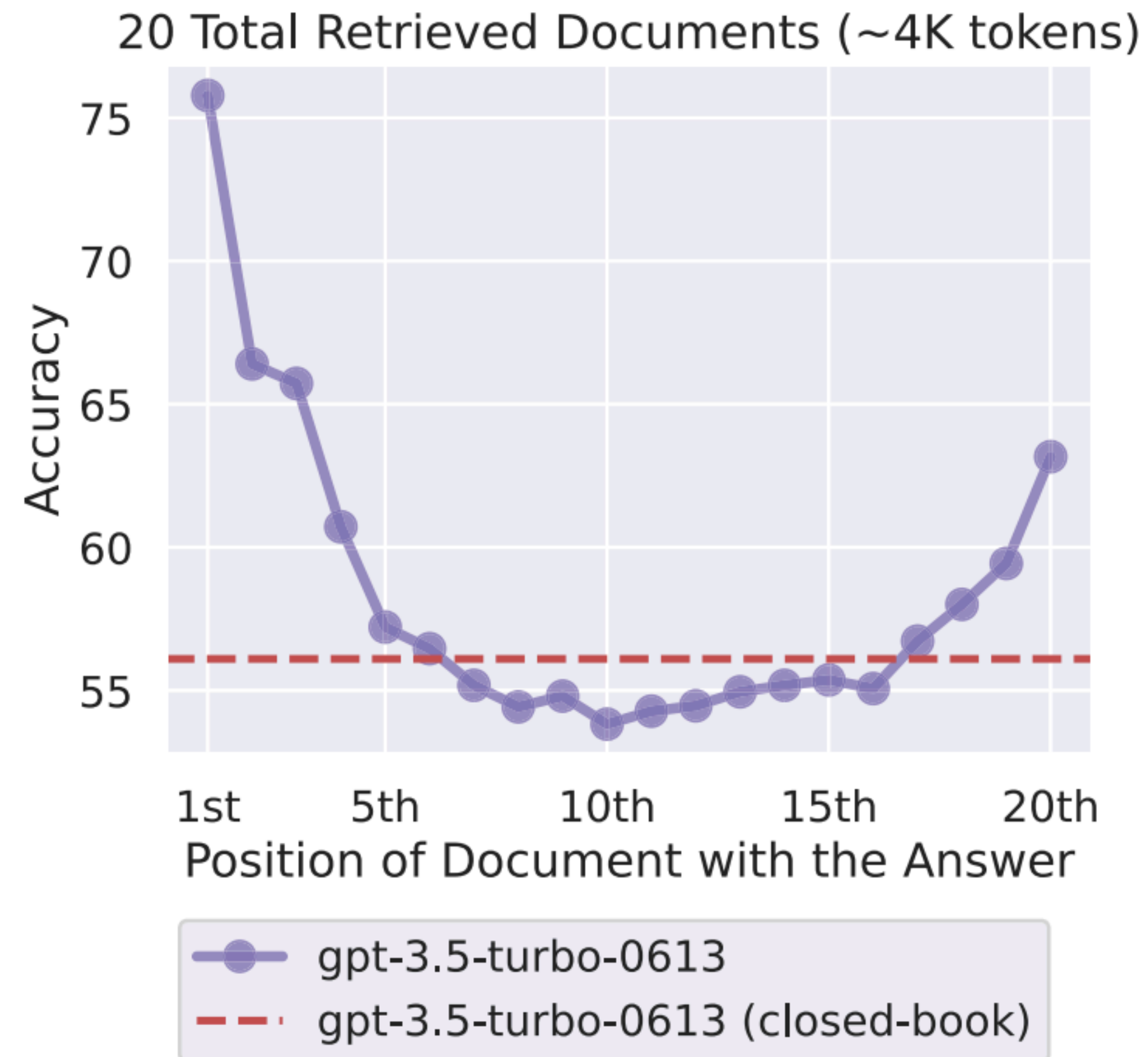
The Code

- Shouldn't take a lot of time (100s max)
- Profile it for variety of inputs
- Always useful to dump any diagnostics data
- Code cannot see the “question”, however:
 - If you control the agent you can send a token which connects “question” to the MCP call
- Can be resource intensive :)
 - The agent call has an associated non-zero cost
 - It will most likely be triggered by human
 - Expect little traffic

PART 4

The output

Context



Liu et al., *Lost in the Middle*

The output

The output must serve the task

The output

Question:

Do these [50] proteins interact?

MCP answer:

**API raw table of size:
 $(50^2)/2 * 10 = 12'500$**

JSON (2x - 4x)

- Agent cannot count...
- Bad AI summary
- Misinterpretation
- Not very useful even if it's parsable
- Very costly!
- Can crash the agent

Output size

Under 2,000 Tokens	Excellent
2,000 - 8,000 Tokens	Reasonable when structured
8,000-20,000 Tokens	Use with caution, grounding may be compromised
Over 20,000 Tokens	Generate summary or a link to a raw file. Do not show the raw data to the agent

The output

Question:

Do these [50] proteins interact?

MCP answer:

Yes.

- Parsable!
- Cheap!
- Not very useful...

The output

Question:

Do these [50] proteins interact?

MCP answer:

Summary:

- Total number of interactions
- Degree per each protein
- Average degree
- Score bins
- Top neighbors per protein

- Scales linearly
- More useful
- Parsable
- Interpretable
- Actionable

The output

≤30 edges: complete interactions, including identifiers and evidence channels

31–100 edges: identifiers and zero-valued evidence omitted

101–500 edges: only protein names and combined scores. Node statistics added

>500 edges: only the top 100 edges. Node statistic added.

Output size

Every MCP output
has to fit within a
reasonable token limit

Under 2,000 Tokens	Excellent
2,000 - 8,000 Tokens	Reasonable when structured
8,000-20,000 Tokens	Use with caution, grounding may be compromised
Over 20,000 Tokens	Generate summary or a link to a raw file. Do not show the raw data to the agent

How to limit output size

- Be creative. The output format doesn't need to be consistent, between the calls (!)
- Remove unnecessary data
- Cut the number of data points:
 - **IMPORTANT:** Inform the data is cut
 - Clearly provide the size of full list
- Provide statistics
- Provide a way to get specific data-points.

Format (just use JSON)

- Output is transmitted as JSON
 - Data doesn't need to be JSON, but...
 - ...the structure needs to be serialized to JSON

```
{
  "structuredContent": {
    "query": "cell",
    "species": 9606,
    "matches": [
      {
        "go_id": "GO:0007049",
        "description": "cell cycle"
      },
      {
        "go_id": "GO:0007155",
        "description": "cell adhesion"
      }
    ],
    "more_matches": true
  }
}
```

Why JSON?

- JSON/dict structure is clear:
 - Each value is explicitly associated with its field
 - Good for nested heterogeneous data
 - Usually it would be a simple structure (but keep them shallow)
- Can be 2x-4x more expensive than TSV

Why TSV?

- Good for homogenous records
- .. with few stable columns
- Token efficient
- Not suitable:
 - Heterogeneous data
 - Wide tables (many columns)
 - Missing values

Probably not XML...

- Less token efficient than JSON
- Nesting increases distance between datapoints
- Zero advantage over JSON for MCP
- **Avoid raw graph serializations (BioPax, RDF/XML)**
 - Related data-points can be on another planet

Graph serializations

Graph serialization is good for storing relationships, but raw graph output is often poor LLM context.

The model must join facts by identifier across separate parts of the graph before it can answer a simple query.

```
:cow      a :Animal .  
:dog      a :Animal .  
:Jupiter  a :Planet .  
:Mars     a :Planet .
```

```
:cow      :colour "brown" .  
:dog      :colour "brown" .  
:Jupiter  :colour "brown" .  
:Mars     :colour "red" .
```

```
[  
  {"name": "cow", "type": "animal", "colour": "brown"},  
  {"name": "dog", "type": "animal", "colour": "brown"},  
  {"name": "Jupiter", "type": "planet", "colour": "brown"},  
  {"name": "Mars", "type": "planet", "colour": "red"}  
]
```


How to limit output size

- Be creative. The output format doesn't need to be consistent, between the calls (!)
- Remove unnecessary data
- Cut the number of data points:
 - **IMPORTANT:** Inform the data is cut
 - Clearly provide the size of full list
- Provide statistics
- Provide a way to get specific data-points.

PART 4.5

Metadata

Metadata

**Metadata is explanatory and actionable
prose attached to the output.**

Metadata

```
{
  "structuredContent": {
    "query": "cell",
    "species": 9606,
    "matches": [
      {
        "go_id": "GO:0007049",
        "description": "cell cycle"
      },
      {
        "go_id": "GO:0007155",
        "description": "cell adhesion"
      }
    ],
    "more_matches": true
  }
}
```

Metadata

```
{
  "structuredContent": {
    "query": "cell",
    "species": 9606,
    "matches": [
      {
        "go_id": "GO:0007049",
        "description": "cell cycle"
      },
      {
        "go_id": "GO:0007155",
        "description": "cell adhesion"
      }
    ],
    "note": "the search text matched more go terms, here we
show only [2]. Do not claim there are only [2] matches."
  }
}
```


Metadata

```
{
  "structuredContent": {
    [...] output structure [...]
    "note": "the search text matched more go
terms, here we show only [2]. Do not claim
there are only [2] matches.

    "next_step": In order to see protein
associated with a specific GO term call
get_proteins_for_go_term tool."
  }
}
```

Metadata

```
{
  "structuredContent": {
    [...] output structure [...]
    "Internal guidance" do not repeat this note
    to the user. The user can inspect the generated
    image, but you should not infer or report
    specific interaction details from the image
    alone. Use `string_interactions_query_set` with
    the same parameters to verify direct or
    indirect interactions."
  }
}
```

No results...

- The site is broken...
- The identifier was not found...
- The identifier was found but there was no data associated with it.
- The identifier was found, there is associated data, but something else is missing...
 - The datapoint is below cut-off
 - The enrichment is not significant

No results...

- The site is broken...
- The identifier was not found...
- The identifier was found but there was no data associated with it.
- The identifier was found, there is associated data, but something else is missing...
 - The datapoint is below cut-off
 - The enrichment is not significant

All of these reasons have completely different meaning for the end users, and are differently actionable.

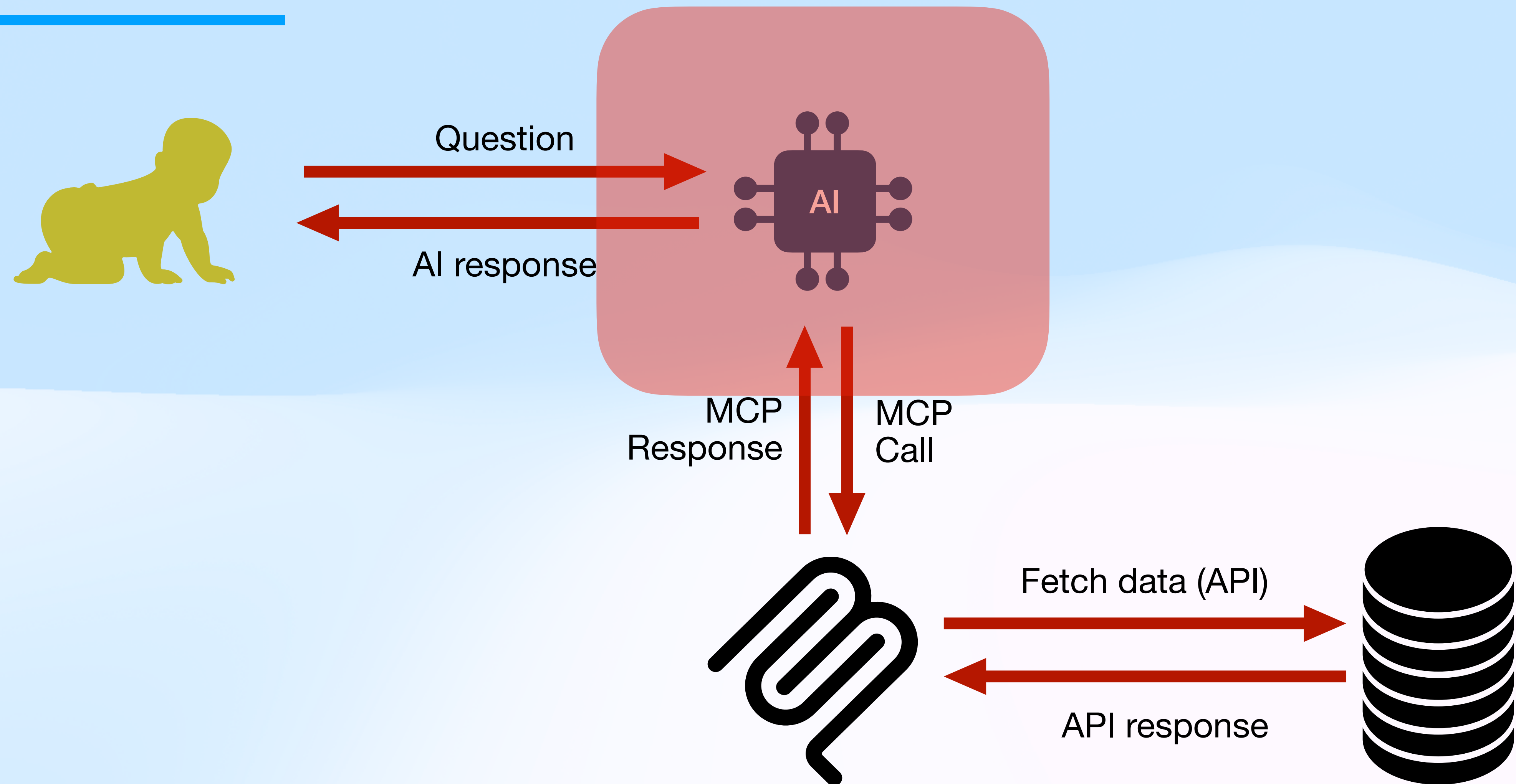
Use metadata for

- Highlight assumptions and defaults
- Communicate automatic changes
- Communicate limitations
- Clarify interpretation
- Suggest next steps
- Instruct how to use output

PART 5

Deployment

Workflow



MCP for scientific usage

Should be designed for non-agentic LLMs

- Wider model and client compatibility
- Lower cost and latency
- Smaller, clearer inputs and outputs
- Fewer transformations and assumptions
- Easier inspection and reproducibility

Testing

- **Always tweak and test with smallest usable models:**
 - GPT-5 nano
 - Gemini 3.1 Flash-Lite
- **If tools work for small models:**
 - definitions are clear
 - behavior will be robust
- **Create a set of test examples:**
 - typical cases
 - edge cases
 - ambiguous queries
 - multi-step workflows
- **Verify that agent**
 - **selected the correct tool**
 - **filled the correct parameters**
 - **Correctly Interpreted the results**
- If possible test with two small models from different vendors (Google, openAI, Anthropic, Deepseek) for added robustness.

System prompt / instructions

Keep the system prompt general. When testing, never use system prompt in order to help interpreting the MCP output.

```
client = OpenAI(api_key=config["OPENAI_API_KEY"])

response = client.responses.create(
    model="gpt-5-nano",
    instructions=system_prompt,
    input=user_input,
    previous_response_id=previous_response_id,
    tools=[
        {
            "type": "mcp",
            "server_label": "mcp-workshop",
            "server_url": https://mcp.string-db.org/,
            "require_approval": "never",
        }
    ],
)
```


Hands-on!